catanrl v2

experiment log

living document · last updated 25 october 2026 · github · wandb

Contents

1 goal and current best bots 2
1.1 current best bots 2
1.2 how to read this log 3
1.3 setup 3
1.4 v1, for reference 4
2 architecture 5
2.1 v1, informal 5
2.2 v2, formal 5
3 the ppo recipe 7
3.1 the recipe 7
3.2 two players 7
3.3 four players 10
3.4 the critic 11
4 search at play time 13
4.1 two players 13
4.2 four players 15
4.3 search options 16
5 hidden information 17
5.1 the hand tracker 17
5.2 predicting the hidden features 19
5.3 open 21
6 alphazero 21
6.1 first runs and cost 21
6.2 the 2p specialist at 2048-sim targets 23
6.3 softer targets 24
6.4 what next 25
7 refitting the critic after ppo 27
7.1 the start's critic 27
7.2 the refit 28
8 benchmark against released canopy 29
8.1 setup 29
8.2 results 30
9 monte carlo playouts instead of a tree 32
10 one model for any number of players and any map 33
10.1 one network over 2–4 players 33
10.2 adding blob maps 34
10.3 next 35
11 trading 35
11.1 is trading worth an action space? 35
11.2 a critic-driven trade harness 37
11.3 learned trading 39
12 how markov a decision is 40
12.1 an lstm over the seat's own decisions 40
12.2 the whole history 42
13 deployment 42
13.1 tables on catan.marcoya.ng 42
13.2 search in the deployed bots 43
14 llm env and eval 43
14.1 frontier models as players 43
14.2 catan as an rl env for language models 44

1 goal and current best bots

the goal is a recipe for the best catan bot out there, for any number of players on any map. v1 (catanrl) got a two-player bot to about 90% against catanatron's `ValueFunctionPlayer` (F). v2 rewrote the stack around a C++ engine and pufferlib, and its raw policy is past 95% in 2p from scratch. the bottom line so far: self-play PPO on a graph network gives a strong raw policy, and a search on top wins 53.4–62.4% of games against that policy. two alphazero runs on top of it made the network worse, not better (Section 6).

most experiments are a config under `configs/experiments/`. a number without an eval record says where it comes from: a training metric, or a json under `report/data/`.

marco's recipe, in two lines. implement the engine in C++, turn it into a pufferlib env, run for about 1B steps, and get a really good shared-trunk critic + policy that beats `ValueFunctionPlayer` about 90% of the time. put the critic + policy inside an MCTS harness, with a simple probabilistic belief-sampling determinization, to get another few points head to head over the raw policy.

measured. the raw policy is at 95% vs F (Section 3). the search edge is 53.4–55.2% with our default PUCT search and 62.4% with gumbel alphazero's root at 2048 sims (Section 4).

latest (25 october 2026). the deployed 1v1 net, continued for 1b steps on openhex's 1v1 rules (friendly robber, balanced dice), beats its start 63.2% and F 98.8%; it is being continued for another 1b steps at its final, lowest learning rate. a first try that restarted the learning rate at its peak fell to ~41% against its start and was stopped (Section 3). the critic refit came back null (Section 7). canopy's search lost to our search 33.0 to 67.0 over 1,000 games (Section 8).

1.1 current best bots

Table 1: the best agent per setting, on held-out boards. 2p scores are win rates; 4p scores are slot 0's score share. v2 trains from scratch (no DAgger) and lands 5 points above v1's warm-started reference in 2p. hidden information is worth under a point in 2p: the fair `public` and belief-averaged nets are level with the raw `full` net. 4p is trained and played at the `private` level, the only fair level past two players.

agent	opponent	games	score	record
2 players, BASE, 15 vp				
raw policy at the `public` level (`puffer-graph2/public@e3600`): the fair raw bot, Section 3	F	5,000	**94.96** 94.3–95.5	`dc8a611a`
`full` net, belief-averaged (`BELIEF`, 16 worlds, `puffer-graph2/full@e3350`): the deployed 1v1 bot	F	5,000	**95.06** 94.42–95.63	`67869e7a`
2 players, openhex's 1v1 rules (friendly robber, balanced dice)				
`ppo-2p-fr/full@e7600`, raw (`full` net continued 1b steps, Section 3)	F	5,000	**98.84** 98.50–99.10	`e435b80e`
the same, raw	its start, `puffer-graph2/full@e3350`	5,000	**63.2** 61.8–64.5	`36bd2384`
`full` net, raw (unfair: sees F's dev cards)	F	5,000	95.16 94.53–95.72	`ff8528a4`
2048-sim gumbel MCTS, one world, on `puffer-graph2/public@e3600` (Section 4)	its own raw policy	1,000	**62.4** 59.4–65.3	`831eea84`
released canopy's raw network (Section 8)	our raw `public` policy	1,000	30.5 27.7–33.4	`1c916131`
released canopy's 1600-sim search (Section 8)	our 2048-sim gumbel MCTS	1,000	33.0 30.2–36.0	`96d94fab`
4 players, BASE, 10 vp (score share, parity 25)				
raw policy (`ppo-4p/main@e3650`, Section 3)	F, F, F	4,000	**78.55** 77.25–79.79	`e5b302ee`
MCTS 1024 sims, one world (`ppo-4p-critic-fix/lambda-critic-g1@e100`, Section 4)	three raw copies	2,000	26.9 25.0–28.9	`657b314d`
trade proposer (`TRADE`, `ppo-4p/main` in every seat; the no-trade control scores 25.0, paired +16.2 pp; Section 11)	three accepting copies	4,000	**41.2** 39.7–42.7	`c4900f87` `7164d396`
one model for 2–4 players and any map (`ppo-mixed/main@e3550`, Section 10)				
2p, BASE	F	5,000	92.3 91.6–93.0	`b2690265`
4p, BASE (share)	F, F, F	4,000	77.0 75.7–78.3	`0f448f68`
4p, blob maps it never trained on (share)	F, F, F	4,000	73.9 72.5–75.3	`7c17d0ae`
v1 reference (graph-s, DAgger + MARL PPO, 73.7M steps), for history; see Section 1.4.1				
raw policy, belief-averaged	F	2,000	89.65 88.24–90.91	v1 report
MCTS 256, determinized	F	1,000	90.5 88.5–92.2	v1 report
MCTS 1024 on the true game (unfair)	F	1,000	92.7 90.9–94.2	v1 report

1.2 how to read this log

sections are ideas, not dates. each experiment is one block:

- a status chip: **done**, **running**, **queued**, **stopped** (ended early), **todo** (planned, nothing run) or **parked** (planned, set aside);
- a meta line: when it ran, whose idea it was, its config, its wandb runs and the eval records that hold its numbers;
- a short summary, then the question, the setup, the result and how to read it;
- the command that reproduces it.

dates appear only in the meta lines. marco's ideas are credited to him. ideas of mine that were never run are marked "(my suggestion, not queued)".

records. an 8-character hex id in backticks (`831eea84`) names an eval record: it is the start of the record's eval id. a record lives under `runs/<experiment>/<variant>/evals/<suite>/`, or under `runs/_evals/<suite>/` for an eval-only row. `catanrl verify <eval id>` replays a record's games and checks them against it. it takes any unique prefix of the id, or the record's path. a record stored in several run dirs (the critic fix's start row `60cd10b6` sits in all four arms) needs the path.

commands. every command runs from the repo root. `catanrl run <config>` runs an experiment's variants, in dependency order, with their finals; `--only <glob>` picks variants. an eval-only row is one `catanrl eval` line, rebuilt from the record's agent specs.

wandb. the meta line links each run on wandb (project link at the top). runs before 2026-10-17 were backfilled from `runs/*/metrics.jsonl` and the eval records. since 2026-10-17 a live mirror (`catanrl.wandb_mirror`) logs every run whose config sets `meta.wandb.project`, and `tools/wandb_backfill.py` re-syncs a run directory into the same wandb run. the record on disk is the source of truth; a wandb failure is only a warning. a run logs losses, entropy, KL, clip fraction, game length, seat win rates, throughput, the select curve and the critic's calibration (`critic/value_sum_*`: the seats' predicted win probabilities summed, which should match the share of games somebody wins). a link labelled "eval only" is an eval run: its wandb page shows no finals, and its results are in the records. a link tagged "no local mirror" is computed from the run's config hash; no local wandb directory confirms that it was synced.

1.3 setup

engine. cppanatron, a C++20 port of catanatron kept in lockstep parity with it: 2–6 players, explicit maps of any shape. the observation encoder, the action codec, the threaded multi-game env, the hand tracker and the batched MCTS all live in C++, so training, search and deployment can't drift apart. the network reads one float32 row per (game, seat), seat-relative and padded to the env's maxima: `[global | players | tiles | nodes | edges | topology | action mask]`.

observation levels. how much of the other seats' hidden state a row shows.

- `private`: what the seat itself sees. the encoder zeroes the other seats' 15 hidden features (5 resource ratios, actual VPs, 5 dev-card counts, 4 playable flags). that is exactly what a sampled world could change, so averaging a `private` policy over sampled worlds gives back the plain argmax policy.
- `public`: in 2p every resource movement is seen by both seats, so the opponent's exact hand is a perfect card counter's knowledge. a `public` policy is fair in 2p without belief averaging.
- `full`: adds the opponent's dev cards in hand. played raw, it is unfair.

agents. each player is a spec string.

- `POLICY:ckpt=<run>@<epoch>`: the raw network, argmax.
- `BELIEF:ckpt=…:worlds=16`: the raw policy averaged over 16 worlds sampled from the hand tracker's belief. it is the fair way to play a network trained on more than its seat sees.
- `MCTS:ckpt=…:sims=N:dets=K`: a batched N-player PUCT search over K worlds sampled from the tracker at the root, with per-seat values backed up $\max^n$ and exact chance nodes for dice, dev-card draws and steals. `oracle=true` searches the true game and is only a reference.
- `F` is catanatron's `ValueFunctionPlayer`; `R` plays uniformly random legal moves.
- `<run>@eN` is the run's checkpoint at epoch N; `@best` is the one its select suite chose, `@latest` the last one saved.

the slot after an F. F (`enemies=next`) robs and blocks the seat after it. a 4p suite seats `lineup[(seat + r) mod 4]`, so slot 0 always moves right after the F in slot 3 and pays for it. any 4p table of the shape `[A, B, F, F]` therefore needs both slot orders, or the two learner slots summed (Section 5 has the diagnosis).

rules. 2p is `base-2p`: the BASE map, 15 vp, discard above 9 cards. 4p is `base-4p`: BASE under the standard rules, 10 vp, discard limit 7, which is also the engine's default. canopy trained with a friendly robber and balanced dice; we use neither.

boards. every number is on boards training never saw; `catanrl validate` checks the training pools against the suites.

- `heldout-2p`: 2,500 boards (seed 67) in both seat orders, 5,000 games.
- `heldout-4p`: 1,000 boards (seed 68) in 4 rotations, 4,000 games.
- `heldout-random-4p`: 1,000 blob-shaped boards of 15–30 tiles (seed 69) in 4 rotations. `heldout-mixed` mixes 2p and 3p BASE with 4p blobs.
- the `hexagon` generator (training only) keeps the BASE shape with shuffled numbers and generated harbors.
- checkpoints are chosen every 50 epochs on separate `select-*` suites, so the held-out number never picked its own checkpoint. `select-2p` is 200 boards (seed 123, 400 games) and `select-4p` 800 games. `select-mixed` is 2, 3 and 4 players on BASE, 100 boards each in every seat rotation (900 games, seed 126).
- the alphazero and critic-refit runs select on `select-2p`'s seed (123) in both seat orders: `select-2p-2k` is 2,000 boards (4,000 games), `select-2p-500` its first 500 boards (1,000 games).

measurement. intervals are wilson 95%, printed small and grey after the number (lo–hi). pp means percentage points. in N-player games the number is slot 0's *score share*: a win counts 1 and a truncated game gives 1/N to every seat, so parity is 1/N. agents that use information a player doesn't have (a raw `full` policy, an oracle search) are marked unfair; everything else is fair.

paired statistics. two rows played on the same game ids pair game by game: mcnemar over the shared ids (`tools/search_probe.py --pair`), with holm when a decision rests on two tests. 4p tables played in both slot orders use a bootstrap over board × rotation pairs. power figures are for 80% power at a two-sided 5% level.

1.4 v1, for reference

1.4.1 v1 reference rows DONE

ran in v1 (catanrl), before v2 · **idea** marco

tl;dr. v1's best 2p numbers, kept for history and never rerun: its belief-averaged raw policy scored 89.65 vs F, and its search beat that policy 51.1–58.1% depending on sims. v2 from scratch is 5 points higher against F.

question. what did v1's best 2p bot score, as a yardstick for v2?

setup. v1 (catanrl): `graph-s`, DAgger from F, then MARL PPO, 73.7M steps. the numbers come from the v1 report and are never paired with v2's.

result.

Table 2: v1's reference rows, from the v1 report.

agent	opponent	games	score
raw policy, belief-averaged	F	2,000	89.65 88.24–90.91
MCTS 256, determinized	F	1,000	90.5 88.5–92.2
MCTS 1024 on the true game (unfair)	F	1,000	92.7 90.9–94.2
MCTS 256 / 1024 / 4096 sims	its belief-averaged raw policy	1,000	51.1 / 55.1 / 58.1

reading. two more v1 results that later sections lean on. v1 never trained alphazero above 512 sims, and its frozen-teacher distillation of an 8 × 64-sim search (EX-1d) gave +1.3 vs F and 50.6% head to head, both null (Section 6). v1 also ran canopy's puct recipe on a 55%-vs-F graph net and raised it to 70.5% raw vs F over 10 iterations, but never from a start above 90% (Section 8).

reproduce

```
# not reproducible in this repo: numbers from the v1 report (github.com/mcrco/catanrl)
```

2 architecture

which network family should carry the policy and the critic? marco's informal v1 ablations pointed at canopy's graph network: the fastest, the most efficient, and the only one that scales to any map, because it predicts board actions per node, edge and tile. he still wanted a formal MLP / LSTM / CNN / GNN ablation, and it agrees: graph ≈ lstm > cnn > mlp at 2p, and the structured head is what matters. every other section uses the graph net `graph-s` (canopy nexus-v3-style message passing, 541,540 parameters; details in Section 3).

2.1 v1, informal

2.1.1 v1's informal ablations DONE

ran in v1 (catanrl, branch arch-ablation) · **idea** marco

> **tl;dr.** in v1 the graph net predicted held-out game outcomes better than an MLP or a CNN, and under full RL the CNN trailed it by 7–12 points vs F at a quarter of the speed. a graph transformer tied message passing, so the structured head matters, not the mixing layer.

question. which family predicts catan best, and at what cost?

setup. imitation loss can't compare architectures: search targets carry the teacher's logit shape, so the loss favours the teacher's family. v1 used a teacher-neutral target instead: predict the terminal outcome of held-out games (win/draw/loss cross-entropy in nats; 0.693 is a coin flip).

result.

Table 3: v1's teacher-neutral outcome prediction on held-out games; lower is better.

model	params	outcome loss (nats)
`graph-s` (message passing)	0.5M	**0.396**
`gtx-s` (graph transformer over the same node and tile tokens)	—	0.397
`mlp-512`	2.4M	0.464
xdim CNN	16.8M	0.83

under the full RL recipe (DAgger, then MARL PPO) a 78.5M-parameter CNN replica trailed `graph-s` by 7–12 points vs F at a quarter of the training speed. `graph-m` tied `graph-s` head to head (50.6%) at 2.3× the cost per step. step cost at batch 512: MLP 3 ms, `graph-s` 26 ms, CNN 120 ms.

reading. the CNN predicted the winner worse than a coin flip, and its value head memorized within a few hundred steps. the graph transformer tying message passing says what matters is the structured head (per-node settlement and city logits, per-edge road logits, per-tile robber logits). the graph net is launch-bound (about 200 small kernels per forward), which CUDA graphs fix at inference.

```
reproduce
# not in this repo: v1 (github.com/mcrco/catanrl), branch arch-ablation
```

2.2 v2, formal

2.2.1 formal ablation at 2p: graph-s, lstm, cnn, mlp DONE

ran 2026-10-18 01:37–13:08 (cnn port built 2026-10-17) · **idea** marco · **config** `configs/experiments/arch-ablation.yaml` · **wandb** graph-s, lstm, cnn, mlp · **records** `039b60f0`, `1cd2de87`, `073da498`, `76c88ebd`, `189e5e8d`, `8e2e23d8`, `5abf6f0c`

> **tl;dr.** four architectures, one recipe, one budget, 2p: `graph-s` 90.7 vs F, `lstm` 90.3 (and a coin flip head to head with `graph-s`), `cnn` 80.4, `mlp` 64.9. the ranking is graph ≈ lstm > cnn > mlp, and the graph net is also the smallest of the four. one seed at 100M steps ranks the families but can't separate graph from lstm.

question. does v1's informal ranking hold under one controlled v2 recipe?

setup. the current recipe (Section 3) at the fair `private` level, so the recurrent cell has hidden information to remember. one seed, 100M agent steps per architecture (757 epochs). checkpoints selected on `select-2p` every 50 epochs; reported on `heldout-2p` vs F and head to head against the `graph-s` cell's best checkpoint on the same boards. 100M steps is enough to separate families: `puffer-graph2` was at 86% vs F by then, and v1's gaps were 7–12 points. every cell has one shared trunk for policy and critic.

- `graph-s`: the recipe's network.
- `lstm`: the `graph-s` trunk plus an LSTM cell over its pooled embedding, stepped at each of the seat's own decisions (Section 12 has the build).
- `cnn`: v1's xdim CNN ported to the v2 row (`src/catanrl/models/cnn.py`). v1's `xdim-default` size: 16.8M parameters per network, 15.1M of them in the projection after the convolutions. the tile, node and edge blocks are gathered onto v1's 11 × 21 plane, laid out once from the BASE map. three deviations from v1: GroupNorm instead of BatchNorm, so rollouts and evals run the same network (`norm: batch` restores v1); every block feature is a channel (47 at 2p) instead of v1's 20 hand-picked planes; and v2's structured heads. BASE map only: the `hexagon` generator has BASE's dims but another tile order, and is refused at launch.
- `mlp`: a flat network over the padded row (`configs/models/mlp.yaml`), BASE dims only.

result.

Table 4: the formal ablation on `heldout-2p`, 5,000 games per number. "vs F" is the argmax policy against F; "vs graph-s" is the same policy against the `graph-s` cell's best checkpoint on the same boards. wall time includes the select evals. records: `graph-s 039b60f0`; `lstm 1cd2de87` / `073da498`; `cnn 76c88ebd` / `189e5e8d`; `mlp 8e2e23d8` / `5abf6f0c`. the same table at 4p, and the `graph-s` cell with a hidden-feature head, are in Section 12 and Section 5.

arch	params	steps/s	wall	select-2p (best)	vs F	vs graph-s
`graph-s`	0.54M	17.6k	1.6 h	92.5 (e750)	**90.7** 89.9–91.5	—
`lstm`	0.86M	14.8k	1.9 h	92.0 (e700)	**90.3** 89.4–91.1	**51.0** 49.7–52.4
`cnn`	16.8M	5.6k	7.5 h	81.5 (e700)	80.4 79.3–81.5	33.7 32.4–35.0
`mlp`	2.2M	108k	0.3 h	68.0 (e700)	64.9 63.6–66.2	18.1 17.0–19.2

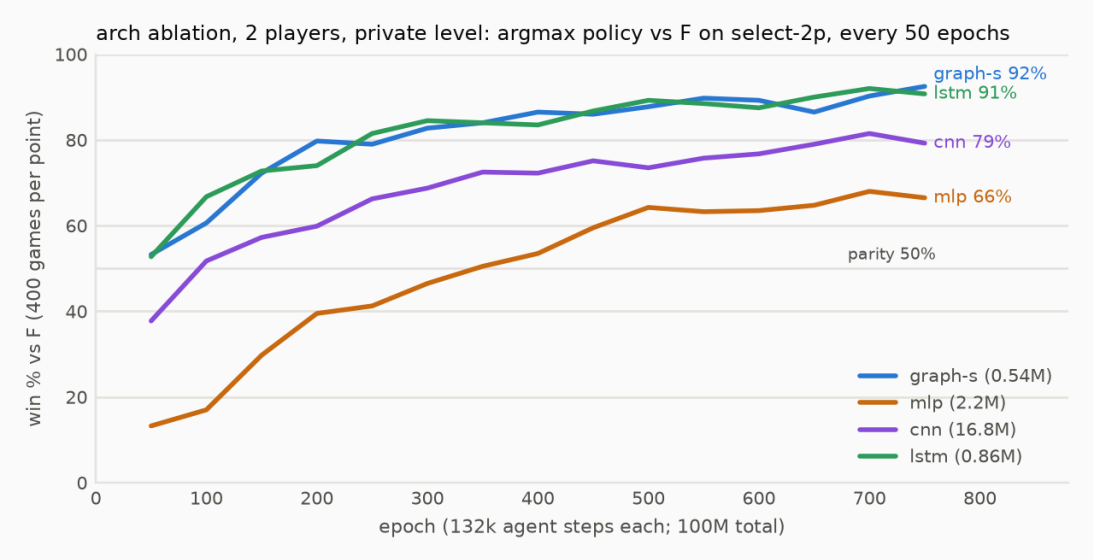


Figure 1: the four cells' selection curves (argmax policy vs F on `select-2p`, every 50 epochs). the graph net and the lstm are indistinguishable from epoch 300 on; the cnn tracks ten points lower; the mlp never catches up.

reading. graph ≈ lstm > cnn > mlp, and the structured head is what matters. the lstm ties the graph net head to head (51.0, a coin flip) at 1.6× the parameters and 0.85× the throughput: at this budget, memory over the seat's own decisions adds nothing the current row lacks (at 4p it does, Section 12). the cnn has 31× the parameters and a third of the speed; it is ten points behind vs F and loses two games in three to the graph net. the mlp is 26 points behind and loses four in five. at 0.3 h the mlp is the cheap baseline; the cnn's 7.5 h is why v1 abandoned it.

reproduce

```
catanrl run configs/experiments/arch-ablation.yaml
# one cell: add --only graph-s (or mlp, cnn, lstm).
# the head-to-head finals of mlp, cnn and lstm need graph-s's best checkpoint, so the runner trains graph-s first.
```

2.2.2 graph vs lstm at the full budget, with more seeds TODO

idea assistant · **config** `configs/experiments/arch-lineage.yaml`

tl;dr. one seed at 100M steps can't separate `graph-s` from the `lstm`; that needs the full 500M-step budget and more seeds. (my suggestion, not queued.)

status. nothing run. the closest config, `configs/experiments/arch-lineage.yaml`, sweeps `graph-s`, `graph-m` and `mlp` over two seeds at 400M steps: 6.5 h a cell, so its 12 cells are about a week of GPU. it ports v1's lineage launch script (`full` level, $\gamma = 0.99$, then a $\gamma = 1$ stage from epoch 3000) rather than the current recipe, and it has no `lstm` cell.

3 the ppo recipe

what training recipe makes the strongest raw policy, and what does it reach? self-play PPO from scratch on the `graph-s` net, v1's graph2 run ported to v2: 95% vs F in 2p whether or not the net sees the opponent's hidden cards, and a 78.55 score share against three F in 4p. the one change since is the critic: lambda-return targets, undiscounted, now the default for every run.

3.1 the recipe

- **network.** `graph-s`: canopy nexus-v3-style heterogeneous message passing (node↔node, node↔tile, node↔edge), hidden 96, 3 layers. one shared trunk feeds the policy head and a per-seat value head: one sigmoid per seat, that seat's win probability. the acting row's prediction fills the waiting seats' values, so GAE on each seat's own trajectory costs one network call per game step. 541,540 parameters. Section 2 compares it with other families.
- **training.** pufferlib 3 PPO with two small patches (losses over acting rows only, a value-loss hook). pure self-play: every seat learns. muon at 5e-3 with a cosine to 5e-4, momentum 0.95; $\gamma = 0.999$ per decision, GAE 0.95; 2 update epochs, clip 0.2, entropy 0.005; prioritized 64-step segments; fp32.
- **batch.** v1's graph2 run exactly: 1032 games × 2 rows × 64 steps = 132,096 agent steps per epoch, 16 optimizer steps per epoch, 3786 epochs. an agent step is one seat's row at one game step, so a 2p decision is two agent steps and a 4p decision four. `configs/experiments/puffer-graph2.yaml` lists every deviation from v1 and why.
- **critic.** lambda-return targets, undiscounted (`critic_targets: lambda`, `critic_gamma: 1` in `configs/train/ppo.yaml`), the default since the critic fix (Section 3.4.1). earlier runs used V-trace targets at $\gamma = 0.999$; their yamls pin those values, so their config hashes and records are unchanged.
- **budgets.** 2p: 500M agent steps, 7.95 h. 4p: 1B agent steps = 249M decisions, 9.3 h. short cells for ablations: 100M agent steps at 2p (about 1.6 h for the graph nets) and 750 epochs = 49M decisions at 4p (about 1.9 h).

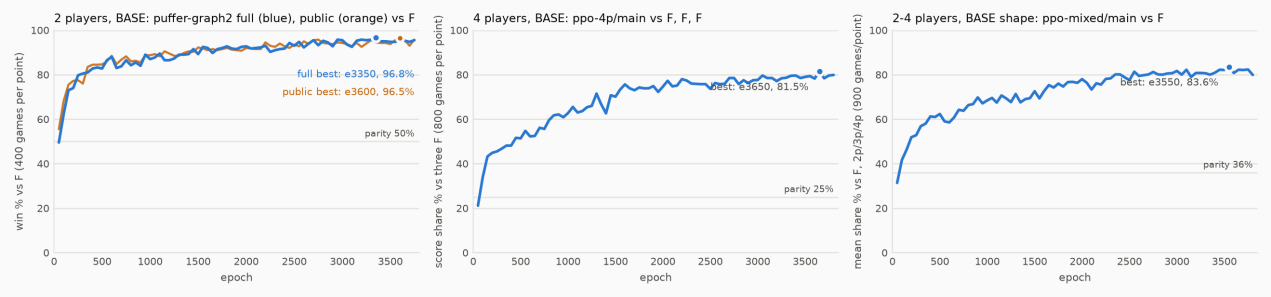


Figure 2: training-time selection curves (argmax policy vs F on the `select-*` suites, every 50 epochs). every run keeps improving through the second half of the cosine, so matched comparisons need the whole budget. the 2p `public` variant (orange) tracks `full` (blue) throughout. per-epoch logs on wandb: puffer-graph2/full, puffer-graph2/public, ppo-4p/main (no local mirror) and ppo-mixed/main (the generalist, Section 10).

3.2 two players

3.2.1 2p at the full level: `puffer-graph2/full` DONE

ran 2026-10-14 12:37–20:34 (training); 2026-10-17 – 2026-10-18 (finals) · **idea** marco · **config** `configs/experiments/puffer-graph2.yaml` · **wandb** puffer-graph2/full · **records** `ff8528a4`, `67869e7a`

tl;dr. v1's graph2 run, reproduced in v2 from scratch at the `full` level: 95.16 vs F played raw (unfair: it sees F's dev cards) and 95.06 belief-averaged (fair). hidden information is worth under a point in 2p, and v2 lands 5 points above v1's warm-started reference.

question. does v2's PPO stack, trained from scratch, reach v1's graph2 result?

setup. the recipe above at the `full` level, graph2's own setting, so the comparison with v1 is like for like. 500M agent steps in 7.95 h; best checkpoint e3350 by `select-2p` (96.8).

result. on `heldout-2p` (5,000 games each), the raw `full` policy scores **95.16** 94.53–95.72 vs F (`ff8528a4`, unfair). belief-averaged over 16 sampled worlds (`BELIEF`) it scores **95.06** 94.42–95.63 (`67869e7a`, fair).

reading. the fair, belief-averaged policy loses nothing against the oracle, which is v1's "hidden information is worth under a point" again. v2 trains from scratch, with no DAgger, and lands 5 points above v1's reference (89.65, Section 1.4.1). this net, belief-averaged, is the deployed 1v1 bot (Section 13). the run's finals also hold two head-to-head search rows, `04fd2c25` and `2aaf2eb7` (Section 4).

```
reproduce
catanrl run configs/experiments/puffer-graph2.yaml --only full
# its finals include the two search rows (MCTS 1024 and 4096 sims), about 11.5 h of GPU on top of training
```

3.2.2 2p at the public level: `puffer-graph2/public`, the fair raw bot DONE

ran 2026-10-18 18:03 – 2026-10-19 02:06 · **config** `configs/experiments/puffer-graph2.yaml` · **wandb** puffer-graph2/public · **records** `dc8a611a`

tl;dr. the same recipe at the `public` level, which in 2p is fair without belief averaging: 94.96 vs F as a plain policy, level with the `full` net. the opponent's dev cards are worth nothing measurable in 2p. one forward pass, no sampled worlds, no tracker: this is the bot to serve raw.

question. can a fair network skip belief averaging and lose nothing?

setup. at the `public` level the net sees the opponent's exact hand, which in 2p is a perfect card counter's knowledge; what `full` adds is only the opponent's dev cards in hand. same recipe and budget as `full`: 7.95 h at 17.8k steps/s. best checkpoint e3600 (select 96.5, against `full`'s 96.8 at e3350; mean over epochs ≥ 3000: 94.4 against 95.1). the two select curves lie on top of each other (Figure 2).

result.

Table 5: what hidden information is worth in 2p: the three observation levels of the same recipe.

agent (vs F, `heldout-2p`)	sees	games	vs F	record
`public` net, raw (`POLICY`, e3600)	fair	5,000	**94.96** 94.3–95.5	`dc8a611a`
`full` net, belief-averaged (`BELIEF`, 16 worlds, e3350)	fair	5,000	95.06 94.42–95.63	`67869e7a`
`full` net, raw (e3350)	F's dev cards	5,000	95.16 94.53–95.72	`ff8528a4`

reading. the one thing `public` can't see, the opponent's dev cards, is worth nothing measurable in 2p. the hidden-feature head reached the same conclusion from the other side: the hand is inferable, the dev cards are not, and the policy didn't need them (Section 5). its search rows (`72819d1a`, `0ef1b81e`, `831eea84`) are in Section 4.

```
reproduce
catanrl run configs/experiments/puffer-graph2.yaml --only public
```

3.2.3 the 1v1 bot on openhex's 1v1 rules, ~1b more steps, judged against its start DONE

ran 2026-10-24 10:46 – 2026-10-25 03:56 · **idea** marco · **config** `configs/experiments/ppo-2p-fr.yaml` · **wandb** ppo-2p-fr/full · **records** `e435b80e`, `6c4f23cb`, `3cd19d23`, `fa618736`, `36bd2384`, `db519a9e`, `8a137f56`, `ffcc5c1a`

tl;dr. openhex's 1v1 queue plays friendly robber and balanced dice; our 2p bots trained on neither. marco: train on friendly robber from now on, continue the deployed 1v1 net for ~1b more steps on it, and judge it against its start (the champion) as well as F. result: 63.2% head to head against its start (~61.7% after correcting for the start's untrained friendly-robber input), 98.8% vs F against the start's 97.3%, still improving at the end.

setup.

- **the rules.** from openhex's source: its 1v1 queue is 15 vp, discard above 9, friendly robber, balanced dice; its 4p queue is 10 vp, discard above 7, normal robber, random dice, which is our 4p setup already. openhex's friendly robber shields an opponent with **public** points ≤ 2. our engine used **actual** points < 3, which counts hidden victory point cards (a mismatch, and the legal moves leak whether the opponent holds one), so the engine now uses openhex's rule (a knight is judged on the points before it takes largest army, as openhex does). balanced dice are in the engine too (marco: "do balanced dice before 1v1 retrain"), following openhex's code: a 36-card deck refilled when 13 or fewer are left, a recency penalty on totals seen in the last 5 rolls,

and per-player seven balancing; search chance nodes use the state's own odds. `base-2p-fr` is openhex's 1v1 exactly: 15 vp, discard above 9, friendly robber, balanced dice. random-dice, normal-robber games play bit-identically to the old engine. the network gets no dice-state input, so the policy plays the dice blind and only the search reads them. the two recorded openhex games say balanced dice but were played on fair dice (an openhex bug, since fixed); the bridge falls back to fair dice if a roll is impossible under the balanced deck.

- **the run.** `puffer-graph2/full@best` (e3350, the deployed 1v1 net) continued on `base-2p-fr` for ~1b agent steps, the recipe otherwise unchanged, at ~17.8k agent steps/s (~17.5 h with monitors and finals).
- **baselines.** the start vs F on `select-2p-fr`: **96.3** 94.9–97.3 (`8a137f56`). its friendly-robber input was never on in training: a copy with that input zeroed beats it **51.5** 50.2–52.9 over 5,000 games (`ffcc5c1a`), so part of any gain over the start is just learning to ignore that input; the final head to head subtracts 1.5 pp for it.
- **identity.** the merge moved the code ids every record is keyed under (catanrl `55ba5984`, engine `a9b40d9c`, train `55588933`), so new rows pair with older records only through `tools/search_probe.py --pair` (old-rule games are bit-identical).
- **the champion eval (marco).** ppo stagnation has been read from the win rate vs F, which sits near its 95% ceiling. every select now also plays the current checkpoint against a champion (here the start), 1,000 games on the same boards each time: ±3.1 pp, paired game by game across checkpoints. the finals include the start under the new rules, since it never trained on them. at e0 the champion row reads 50.0 (`fa6ea360`), as it must.

result. trained 2026-10-24 10:46 to 2026-10-25 ~03:25 (7,600 epochs, 1.0b agent steps at ~17.8k/s), finals by 03:56. the champion row climbed the whole way and was still rising at the end; best is the last checkpoint, e7600.

Table 6: the monitors during training, `select-2p-fr`, mean of ten selects (10,000 games) per block. the last block, e7000–7600, reads 63.2–64.4 vs the start and 98.4–98.6 vs F.

epochs	0–450	1000–1450	2000–2450	3000–3450	4000–4450	5000–5450	6000–6450
vs the start	49.5	51.2	54.4	56.1	57.9	60.5	61.9
vs F	96.5	97.2	97.3	97.6	98.2	98.0	98.2

Table 7: the finals. the start is `puffer-graph2/full@e3350`, the deployed 1v1 net, on the same rules and boards.

final on `heldout-2p-fr` (5,000 games each)	e7600	the start	paired
POLICY vs F	**98.84** 98.50–99.10 (`e435b80e`)	97.32 96.83–97.73 (`3cd19d23`)	+1.52 pp, 126 vs 50, p ≈ 1e-8
BELIEF (16 worlds) vs F	**98.76** 98.41–99.03 (`6c4f23cb`)	97.30 96.81–97.71 (`fa618736`)	+1.46 pp, 127 vs 54, p ≈ 6e-8
POLICY, e7600 vs the start	**63.2** 61.8–64.5 (`36bd2384`)	—	≈ 61.7 after the 1.5 pp input correction
BELIEF, e7600 vs the start	**62.2** 60.9–63.5 (`db519a9e`)	—	≈ 60.7 after the correction

reading. another billion steps on openhex's 1v1 rules made the deployed 1v1 net clearly stronger: about 62% head to head against its own start after correcting for the start's never-trained friendly-robber input (roughly +85 elo), and its losses to F more than halved (2.7% ⟶ 1.2%). the champion row was still rising when the learning rate reached its floor, so the net had not stopped improving; another continuation would likely gain more. none of this is measured inside the search yet: the deployed openhex bots run MCTS-1024 and BELIEF on the old engine and the old net.

reproduce
```
PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True catanrl run configs/experiments/ppo-2p-fr.yaml
```

3.2.4 the same net, another ~1b steps from e7600 at the floor learning rate RUNNING

ran 2026-10-25 18:49 onward · **idea** marco · **config** `configs/experiments/ppo-2p-fr3.yaml` · **wandb** ppo-2p-fr3/full

tl;dr. Section 3.2.3 was still improving against its start when its learning rate hit the floor, so marco asked for another billion steps. the first try restarted the learning rate at its peak and went backwards (Section 3.2.5), so this run simply carries on at the floor, constant 0.00025, from e7600.

question. does the 1v1 net keep improving on openhex's 1v1 rules if training just continues where `ppo-2p-fr` ended?

setup. `ppo-2p-fr` exactly, initialized from `ppo-2p-fr/full@e7600` with a fresh optimizer. the learning rate is held at `ppo-2p-fr`'s floor, 0.00025, for 7,600 epochs (~1b agent steps); there is no cosine. the champion row plays every 50th checkpoint against e7600 (1,000 games, ±3.1 pp, paired across checkpoints; ±1 pp per 10-checkpoint block). vs F is measured alongside. finals on `heldout-2p-fr`, 5,000 games each: `@best` vs F, `@best` vs e7600, and `@best` vs the

original deployed net `puffer-graph2/full@e3350`, each as POLICY and as BELIEF. at e0 it reads 50.0 vs the champion and 98.8 vs F, and the first updates run at kl 0.0006–0.0009, where `ppo-2p-fr` ended. ~17 h.

result. running.

```
reproduce
PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True catanrl run configs/experiments/ppo-2p-fr3.yaml
```

3.2.5 the first try: a fresh cosine from the peak learning rate STOPPED

ran 2026-10-25 14:03 – 18:48 (stopped at e2010) · **idea** marco · **config** `configs/experiments/ppo-2p-fr2.yaml` · **wandb** ppo-2p-fr2/full

tl;dr. the continuation restarted the learning rate at its peak, 0.0025, and fell to 40–43% against its own start within 50 epochs. it stayed there through e2000, so marco stopped it and restarted at the floor (Section 3.2.4).

question. the same as Section 3.2.4.

setup. `ppo-2p-fr` exactly, initialized from `ppo-2p-fr/full@e7600`, with a fresh optimizer and a fresh cosine (0.0025 ⟶ 0.00025 over 7,600 epochs), as `ppo-2p-fr` had done from its own start. the monitors and planned finals were the same as Section 3.2.4.

result. stopped at e2010 (26%), before any finals. the champion row fell to 43.5 by e50 and stayed at 40–43. the 10-checkpoint block means are 43.3, 41.2, 40.6, 42.1 and 40.0, each over 10,000 games (±1 pp). vs F fell from 98.8 to ~97.7.

reading. the learning-rate restart caused this, not the extra steps:

- e7600 ended at the floor learning rate (0.00025): kl 0.001 per update, entropy 0.048.
- the restart at 0.0025 pushed kl to 0.02–0.03 at once, about 20 times larger.
- `ppo-2p-fr`'s own restart from e3350 (entropy 0.084, a broader policy) ran at kl 0.015 and stayed at 48–51 early.

a sharp policy that ended at a low learning rate loses ~8–10 pp head to head if the learning rate jumps back to the peak. the cosine would have returned to the floor by e7600, so this run's end would have tested a warm restart, not plain continued training. lesson: set a continuation's learning rate from its start's final kl and entropy.

```
reproduce
PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True catanrl run configs/experiments/ppo-2p-fr2.yaml
```

3.3 four players

3.3.1 4p specialist: `ppo-4p/main` DONE

ran 2026-10-14 20:55 – 2026-10-15 06:16 · **idea** marco · **config** `configs/experiments/ppo-4p.yaml` · **wandb** ppo-4p/main (no local mirror) · **records** `e5b302ee`, `b07e5bd9`, `95570c74`

tl;dr. the 2p recipe on 4p BASE at the `private` level, the only fair level past two players: a 78.55 score share against three F (parity 25) and 100.0 against three random players. its V-trace critic was biased high, which the critic fix below corrects.

question. does the 2p recipe carry over to four players unchanged?

setup. `puffer-graph2`'s recipe on `base-4p` (BASE, 10 vp, discard limit 7), with all four seats learning. `private` level. 1B agent steps = 249M decisions (3800 epochs), about the decisions of the 2p run; 9.3 h. best checkpoint e3650 by `select-4p`.

result.

Table 8: the 4p specialist. parity is 25 per slot.

lineup (`heldout-4p`, 4,000 games each)	share of the policy's slots	record
policy vs F, F, F	**78.55** 77.25–79.79	`e5b302ee`
two copies vs F, F	93.2 together (40.45 + 52.75)	`b07e5bd9`
policy vs R, R, R	100.0 99.9–100.0	`95570c74`

reading. the two identical copies split 40.45 / 52.75 because of the slot order, not because they differ: the copy in slot 0 sits right after an F (Section 1, Section 10). the critic summed its four seats' win probabilities well above the realized sum, a bias from V-trace targets on waiting rows (Section 3.4.1).

```
reproduce
catanrl run configs/experiments/ppo-4p.yaml
```

3.4 the critic

3.4.1 the 4p critic fix: lambda-return targets, undiscounted DONE

ran 2026-10-18 14:58–16:00 · **config** `configs/experiments/ppo-4p-critic-fix.yaml` · **wandb** lambda, lambda-critic-g1, lambda-g1, lambda-adamw · **records** `60cd10b6`, `7d9374ed`, `f4be1c20`, `1d639013`, `8e93695a`

tl;dr. `ppo-4p`'s four seats' win probabilities summed to about 1.2 against a realized 0.87. lambda-return critic targets put the sum on the truth within about 20 epochs, and undiscounted the sum is 1.00; the policy doesn't move. it is now the default for every run.

question. why is the 4p critic biased high, and what fixes it without touching the policy?

diagnosis. the seat sum was about 1.2 against a realized 0.87 at epoch 300, and 0.93 against 0.90 by the end. cpu replications found two causes.

- V-trace ratios on waiting rows. only the acting seat's row gets $\min(\rho, 1)$; the other three get 1. so each seat's target values the game as if that seat alone played the improved policy, and the four one-sided gains compound through bootstrapping.
- muon's fixed-size update of the 1×128 value output layer. the predicted sum alternates high and low every optimizer step, and the rollout network lands on the high side every epoch.

setup. 100-epoch fine-tunes of `ppo-4p/main@latest` at its final learning rate (5e-4, constant), with lambda-return critic targets (every ratio 1; the policy's advantages stay V-trace's). four arms of about 15 minutes each:

- `lambda`: $\gamma = 0.999$ for policy and critic.
- `lambda-critic-g1`: the critic undiscounted, the policy at $\gamma = 0.999$.
- `lambda-g1`: policy and critic undiscounted.
- `lambda-adamw`: `lambda` with the value output layer on AdamW instead of muon.

result.

Table 9: the critic-fix arms at the end of the 100-epoch fine-tune. seat sum: the four seats' predicted win probabilities summed, against the realized sum. select-4p: the policy's share vs F, F, F, n=800 per point (±2.8). the start row's record sits in all four arms' run dirs, so `catanrl verify` needs its path, e.g. `runs/ppo-4p-critic-fix/lambda/evals/select-4p/60cd10b66aea063c.json`.

arm	critic targets	seat sum vs realized	select-4p	record
start (`ppo-4p/main`)	V-trace, $\gamma = 0.999$	0.93 vs 0.90	79.9	`60cd10b6`
`lambda`	lambda, $\gamma = 0.999$	0.906 vs 0.899	80.4	`7d9374ed`
`lambda-critic-g1`	lambda, critic $\gamma = 1$	0.998 vs 1.000	79.9	`f4be1c20`
`lambda-g1`	lambda, policy and critic $\gamma = 1$	1.000 vs 1.000	80.4	`1d639013`
`lambda-adamw`	lambda, $\gamma = 0.999$, output layer on AdamW	0.906 vs 0.898	79.1	`8e93695a`

every arm reaches its realized sum within about 20 epochs and stays there (Figure 3); `lambda` started at 0.920 against 0.914 in its first epochs. the rollout-vs-trained gap falls from 0.004 to 0.000 in all four arms. per-slot calibration at the end of `lambda-critic-g1`: the seat to move predicted 0.284 against a realized 0.276, the other three within 0.004. clip fraction, entropy and game length (79–80 turns) don't change.

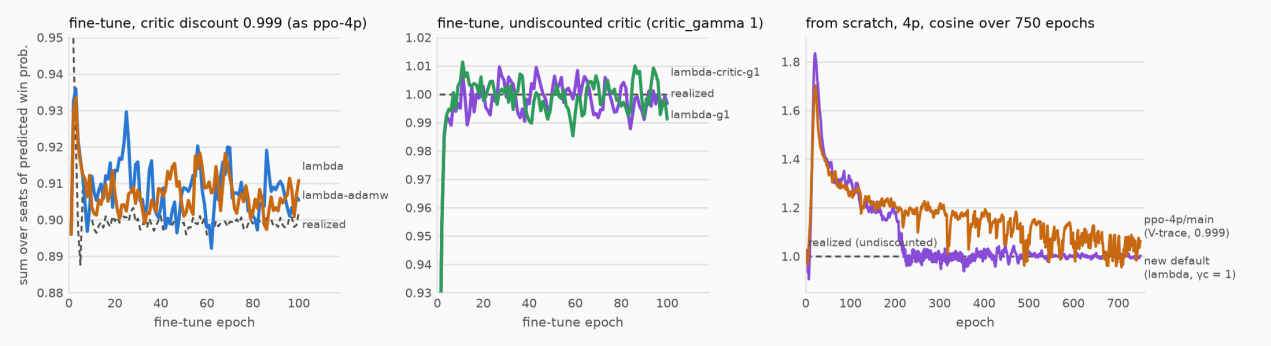


Figure 3: left, `ppo-4p-critic-fix`: the four seats' predicted win probabilities summed, per fine-tune epoch, against the realized sum (dashed: about 0.90 at $\gamma = 0.999$, 1 undiscounted). lambda-return targets put the sum on the truth in every arm within about 20 epochs; the AdamW output layer changes nothing. right, from scratch under the new default (Section 3.4.2): `aux-hidden-4p/plain` (purple) against `ppo-4p/main`'s V-trace critic over its first 750 epochs (orange; its own realized sum is about 0.90, the dashed 1.0 is the undiscounted critic's). both overshoot early; the lambda critic is on its realized sum from epoch 220, while the V-trace critic is still 0.16 above its own at epoch 750.

reading. the targets alone fix it. they also remove muon's phase lock, so the AdamW arm buys nothing. the undiscounted policy plays games of the same length, so $\gamma = 1$ changes nothing the policy optimises at this horizon. per-seat win probabilities that sum to one are what max^n backups in the search want, so `critic_targets: lambda` and `critic_gamma: 1` became the defaults in `configs/train/ppo.yaml`.

reproduce

```
catanrl run configs/experiments/ppo-4p-critic-fix.yaml
# four arms, about 15 min each; each fine-tunes ppo-4p/main@latest, so ppo-4p must be trained first
```

3.4.2 the lambda critic from scratch DONE

ran 2026-10-19 · **config** `configs/experiments/aux-hidden-4p.yaml` · **wandb** aux-hidden-4p/plain, aux-hidden-4p/aux, lstm-4p/main

tl;dr. trained from scratch, the undiscounted lambda critic overshoots early (a seat sum of 1.83 at epoch 21) and is on the realized 1.00 by epoch 220, where it stays. `ppo-4p`'s V-trace critic overshoots too and never comes back (0.94 against 0.90 at epoch 3800). the fine-tunes couldn't show the transient, since their critics started trained.

question. does the new default calibrate when the critic starts from random weights?

setup. the first from-scratch runs under the new default: the two `aux-hidden-4p` arms, `ppo-4p`'s recipe at 750 epochs (49.2M decisions). the arms are a hidden-information experiment (Section 5); here only their critics are read, against `ppo-4p/main`'s first 750 epochs. `lstm-4p` (Section 12) repeats the check on the recurrent net. the numbers are the training metric `critic/value_sum_4p`, not eval records.

result.

Table 10: the four seats' predicted win probabilities summed, by epoch, from random weights. the undiscounted lambda critic's realized sum is 1.00; the V-trace critic's is about 0.90.

run	critic	early overshoot	on the realized sum	later
`aux-hidden-4p/plain`	lambda, $\gamma = 1$	1.83 at e21, 1.39 at e50, 1.31 at e100, 1.18 at e200	1.00 by e220	mean 1.000 over e300–750, within 0.06 at every epoch
`aux-hidden-4p/aux`	lambda, $\gamma = 1$	—	—	mean 0.99
`lstm-4p/main`	lambda, $\gamma = 1$	1.59 at e21	1.00 by e250	mean 1.000 over e300–750
`ppo-4p/main`	V-trace, $\gamma = 0.999$	1.27 at e100, 1.21 at e300, 1.09 at e500, 1.06 at e750	never	0.94 against a realized 0.90 at e3800

reading. the early overshoot is a from-scratch transient, and both target rules produce it; only the lambda critic recovers. it lands at epoch 220, while the learning rate is still four fifths of its peak. explained variance is 0.82–0.85 at the end, as before. the right panel of Figure 3 shows the two curves.

reproduce

```
catanrl run configs/experiments/aux-hidden-4p.yaml --only plain
```

4 search at play time

marco's recipe has a second step: put the trained policy and critic inside a search, sample the hidden cards from the hand tracker, and get about another 5% head to head over the raw policy. this section measures that edge in 2p and 4p, and tests two search options built for the deployed bot. in 2p a PUCT search with one sampled world wins 53–55% against the raw policy, and the 2048-sim gumbel search wins 62.4%, the strongest 2p player we have. in 4p the edge is about two points of share (8% relative), and tree reuse and transpositions change nothing measurable.

the agents. `MCTS:ckpt=…:sims=N:dets=K` is a batched N-player PUCT search. each seat's value is backed up separately ($\max^n$), dice, dev-card draws and steals are exact chance nodes, and the root searches K worlds sampled from the hand tracker (`dets=K`). every fair row below uses `dets=1`, one sampled world per search. `oracle=true` searches the true game, sees the hidden cards, and is only a reference. `root=gumbel` swaps the PUCT root for gumbel alphazero's root over `gumbel_m` candidate moves. `BELIEF:ckpt=…:worlds=16` is the raw policy averaged over 16 sampled worlds, the fair way to play a network trained at the `full` level. `POLICY` is the raw policy's argmax.

4.1 two players

4.1.1 MCTS on the true game vs F DONE

ran 2026-10-14 · **config** `configs/experiments/puffer-graph2.yaml` · **wandb** puffer-graph2/full · **records** `3fdae985`, `609e0aa7`

> **tl;dr.** an oracle search (it sees the opponent's hidden cards) scores 94.8 against F at 64 sims and 95.2 at 256, no better than the raw policy's 95.16. against F a strong policy is already at its ceiling, so every later search row is measured head to head against a raw policy.

question. does search move the number against F?

setup. `MCTS` with `oracle=true` and `dets=1` on `puffer-graph2/full@e3350`, against F on the first 250 boards of `heldout-2p` in both seat orders: 500 games a row. these were finals of `puffer-graph2/full` under the config's first version (`4a56469`), since dropped from the yaml; the eval lines below replay them.

result.

Table 11: oracle search with `puffer-graph2/full@e3350` against F on `heldout-2p`.

agent	sees	games	vs F	record
raw policy, `full` obs (reference, Section 3)	hidden cards	5,000	95.16 94.53–95.72	`ff8528a4`
MCTS 64 sims, oracle	hidden cards	500	94.8 92.5–96.4	`3fdae985`
MCTS 256 sims, oracle	hidden cards	500	95.2 93.0–96.8	`609e0aa7`

reading. all three rows sit inside each other's intervals. vs F cannot see a search gain at this strength, so the search edge is measured head to head below.

reproduce

```
# finals dropped from puffer-graph2.yaml; the records' specs with the defaults left out
catanrl eval "MCTS:ckpt=puffer-graph2/full@e3350:sims=64:dets=1:oracle=true" F \
  --suite heldout-2p --boards 250
catanrl eval "MCTS:ckpt=puffer-graph2/full@e3350:sims=256:dets=1:oracle=true" F \
  --suite heldout-2p --boards 250
```

4.1.2 search edge over the belief-averaged policy (`full` net) DONE

ran 2026-10-17 – 2026-10-18 01:31 · **idea** marco · **config** `configs/experiments/puffer-graph2.yaml` · **wandb** puffer-graph2/full · **records** `04fd2c25`, `2aaf2eb7`

> **tl;dr.** fair search (one world sampled from the tracker) beats the same network's belief-averaged policy 53.4% at 1024 sims and 55.2% at 4096. the edge is real and grows with sims, a point or two under v1's over a stronger raw policy. it cost 11.5 gpu hours for 1,500 games.

question. marco's recipe claims about 5% head to head from search. does v2 get it?

setup. `MCTS:sims=N:dets=1` against `BELIEF:worlds=16` of the same checkpoint, `puffer-graph2/full@e3350`, on `heldout-2p`: 500 boards in both seat orders at 1024 sims, 250 boards at 4096. these are the last two finals of `puffer-graph2/full`, played with 128 games in flight.

result.

Table 12: MCTS against the same network's `BELIEF` policy, head to head on `heldout-2p`. v1's column is its own network against its own belief-averaged policy on 1,000 games.

sims	games	MCTS vs BELIEF	v1, same pairing	record
256	—	—	51.1	—
1024	1,000	**53.4** 50.3–56.5	55.1	`04fd2c25`
4096	500	**55.2** 50.8–59.5	58.1	`2aaf2eb7`

both seats agree: at 1024 sims the searcher wins 55.0 from the first seat and 51.8 from the second.

reading. the edge is there and grows with sims. it is a point or two smaller than v1's, over a stronger raw policy. the cost, 11.5 h of gpu for 1,500 games, is what parks alphazero at these sim counts (Section 6).

reproduce

```
# trains (8 h), then plays every final
catanrl run configs/experiments/puffer-graph2.yaml --only full
# the two search finals alone:
catanrl eval "MCTS:ckpt=puffer-graph2/full@e3350:sims=1024:dets=1" \
  "BELIEF:ckpt=puffer-graph2/full@e3350:worlds=16" --suite heldout-2p --boards 500
catanrl eval "MCTS:ckpt=puffer-graph2/full@e3350:sims=4096:dets=1" \
  "BELIEF:ckpt=puffer-graph2/full@e3350:worlds=16" --suite heldout-2p --boards 250
```

4.1.3 search edge over the raw `public` policy DONE

ran 2026-10-19 02:07–07:10 · **config** `configs/experiments/puffer-graph2.yaml` · **wandb** puffer-graph2/public · **records** `72819d1a`, `0ef1b81e`

tl;dr. the same search over the fair `public` network beats its raw policy 54.9% at 1024 sims and 55.2% at 4096: the same edge the `full` network has. it took half the gpu time, because the opponent is one forward pass instead of sixteen.

question. does the edge hold for the network we would serve raw?

setup. in 2p every resource movement is seen by both seats, so the opponent's exact hand is public, and a `public`-level policy is fair without belief averaging. `puffer-graph2/public@e3600` is the same recipe and budget as `full` at that level (Section 3). `MCTS:sims=N:dets=1` against `POLICY` of the same checkpoint on `heldout-2p`, the same boards as the `full` rows, 256 games in flight.

result.

Table 13: MCTS with `puffer-graph2/public@e3600` against its own raw policy on `heldout-2p`.

sims	games	MCTS vs raw policy	wall	record
1024	1,000	**54.9** 51.8–58.0	1.5 h	`72819d1a`
4096	500	**55.2** 50.8–59.5	3.6 h	`0ef1b81e`

both seats agree. at 1024 sims the searcher wins 60.6 in seat 0 and 49.2 in seat 1, the raw policy 50.8 and 39.4: seat 0 wins about 56% of these games whoever sits there.

reading. the same edge as the `full` network over its belief-averaged self (53.4 and 55.2). the 1,500 games took 5.0 h against the `full` rows' 11.5 h: a one-forward opponent, and 256 games in flight instead of 128.

reproduce

```
# trains (8 h), then plays every final
catanrl run configs/experiments/puffer-graph2.yaml --only public
# the two search finals alone:
catanrl eval "MCTS:ckpt=puffer-graph2/public@e3600:sims=1024:dets=1" \
  "POLICY:ckpt=puffer-graph2/public@e3600" --suite heldout-2p --boards 500
catanrl eval "MCTS:ckpt=puffer-graph2/public@e3600:sims=4096:dets=1" \
  "POLICY:ckpt=puffer-graph2/public@e3600" --suite heldout-2p --boards 250
```

4.1.4 gumbel search at 2048 sims vs the raw policy DONE

ran 2026-10-21 18:56–22:50 · **config** `configs/experiments/gumbel-edge-2p.yaml` · **wandb** gumbel-edge-2p, eval only · **records** `831eea84`

tl;dr. the `public` network inside a 2048-sim gumbel search wins 62.4% of 1,000 games against its own raw policy, far above PUCT over the same policy (54.9 and 55.2). it is the strongest 2p player we have, and the baseline that every later search-harness row pairs with.

question. alphazero's 2048-sim targets made the raw policy worse (Section 6.2.1). are the moves behind those targets good ones?

setup. `MCTS:ckpt=puffer-graph2/public@e3600:sims=2048:dets=1:root=gumbel:gumbel_m=16`, the fair search alphazero trained on, against `POLICY` of the same checkpoint. the first 500 boards of `heldout-2p` in both seat orders, 1,000 games.

result. the search wins **62.4** 59.4–65.3 (`831eea84`). PUCT over the same policy wins 54.9 at 1024 sims and 55.2 at 4096 (Section 4.1.3).

reading. the search had plenty to teach, and the raw policy still got worse learning it (Section 6). these 1,000 games became the search harness, marco's rule for judging a new network: put it in this same search against the same raw start, and pair it game by game with `831eea84` (mcnemar over the shared game ids, `tools/search_probe.py --pair`). the alphazero checkpoints (Section 6), the critic refit (Section 7) and canopy's search (Section 8) are all judged this way.

```
reproduce
catanrl run configs/experiments/gumbel-edge-2p.yaml
```

4.2 four players

4.2.1 4p search vs three raw copies DONE

ran 2026-10-18 16:33–17:45 (400 games); 2026-10-19 07:38–10:15 (2,000 games) · **config** `configs/experiments/search-4p.yaml`, `configs/experiments/search-4p-2000.yaml` · **wandb** search-4p, eval only, search-4p-2000, eval only · **records** `90a1f06c`, `657b314d`

tl;dr. in 4p, MCTS at 1024 sims with one world takes a 26.9 share against three copies of its own raw policy (parity 25) over 2,000 games. that is a two-point edge, 8% relative, about the same relative edge as in 2p. the searcher is ahead at every seat, but the interval just touches parity (p = 0.053).

question. does the search edge carry over to 4p?

setup. `MCTS:sims=1024:dets=1` with the critic-fix network `ppo-4p-critic-fix/lambda-critic-g1@e100` (undiscounted per-seat values that sum to one, the same raw policy as `ppo-4p` to within noise; Section 3), against three copies of that network's raw policy on `heldout-4p`. first 100 boards × 4 rotations (400 games, 71 min), then 500 boards × 4 rotations (2,000 games, 2.6 h with 512 in flight, 4.7 s a game).

result.

Table 14: MCTS 1024 sims, one world, against three copies of its own raw policy on `heldout-4p`. share of wins, parity 25.

table	searcher's share	raw copies' shares	record
100 boards × 4 (400 games)	27.0 22.9–31.6	24.3 (mean)	`90a1f06c`
500 boards × 4 (2,000 games)	**26.9** 25.0–28.9	25.2 / 22.7 / 25.3 (24.4 pooled)	`657b314d`

in the 2,000-game table the searcher won 538 games, p = 0.053 against 25% by the record's own test. its first 100 boards replay the 400-game record (400 of 400 games identical), and the other 1,600 games alone give 26.9. the searcher beats the raw copies' mean at every seat: +3.4 in seat 0, +2.1, +3.3 and +1.3 in seats 1–3. the first seat wins 27.9% of these games overall against seat 2's 23.1, so a small first-seat advantage sits in these numbers too (no F at the table, so no F targeting).

reading. a two-point edge, 8% relative, measured to about ±2. that is the same relative edge as in 2p (7–10% there): real but small at 1024 sims and one world, and a long way from paying for its cost (each searcher decision is 1024 forwards).

```
reproduce
catanrl run configs/experiments/search-4p.yaml
catanrl run configs/experiments/search-4p-2000.yaml
```

4.3 search options

4.3.1 tree reuse, transpositions and leaf dedup DONE

ran 2026-10-20 04:24–14:33 · **idea** marco · **config** `configs/experiments/mcts-reuse-2p.yaml`, `configs/experiments/mcts-reuse-4p-both.yaml`, `configs/experiments/mcts-reuse-4p.yaml` · **wandb** mcts-reuse-2p, eval only, mcts-reuse-4p-both, eval only, mcts-reuse-4p, eval only · **records** `aca296e2`, `6601df9a`, `6010fedf`, `72819d1a`, `90a1f06c`, `12f704f2`, `1b01f4ac`

tl;dr. marco asked for tree reuse and leaf deduplication for the deployed bot. both search options are correct and cost nothing measurable in strength, but they buy nothing either at 1024 sims and one world. every paired row leans one to four points negative, none significantly: a null with a slight unfavourable tilt.

question. does keeping a seat's tree across its own moves, or sharing nodes between move orders, make the same search budget stronger?

setup. three mechanisms:

- `reuse=true` keeps a seat's search alive across its decisions and re-roots it when the game advanced by exactly its own action. the retained visits get a fresh simulation budget on top. about a third of a seat's decisions re-root in 4p and half in 2p; the rest follow a roll, another seat's move or a forced move.
- `transpositions=true` gives each world a transposition table: positions reached by different build, buy or discard orders share one node. the discard chain after a 7 collapses from a tree of 86 nodes to a lattice of at most fifty, and a search holds from a handful to over a hundred such links, by position. the simulation count is unchanged; the search gets deeper rather than smaller.
- leaf dedup evaluates byte-identical rows once. it is bitwise neutral, so it has no key. rows forwarded to the network fall to 46–91% of the total, depending on worlds and players.

both options are keyed only when set, so every stored search record rebuilds unchanged. each row replays the games of a stored search table and pairs with it on shared game ids (`tools/search_probe.py --pair`).

result.

Table 15: MCTS 1024 sims, one world, with the options against the stored row without them, on the same games. 2p is win rate against the raw `public` policy; 4p is share against three raw copies of the critic-fix net.

row	games	with options	stored row	paired	won only with / without	p
2p, both options (`aca296e2` vs `72819d1a`)	200	58.5 51.6–65.1	59.5	−1.0 pp	7 / 9	0.80
4p, both options (`6601df9a` vs `90a1f06c`)	200	25.0 19.5–31.4	28.5	−3.5 pp	8 / 15	0.21
4p, reuse only (`6010fedf` vs `90a1f06c`)	400	25.3 21.2–29.7	27.0	−1.75 pp	9 / 16	0.23

two engine bugs surfaced on the way, and both are fixed. re-rooting treated a steal with one possible card in the sampled world as deterministic, which crashed both 4p rows (engine `b596130`, with a regression test). with both options on, several root actions could share one child through a transposition, and the new root counted that child once per action (`e9f12af`, a no-op with transpositions off, so the reuse-only row is untouched). two records from before `e9f12af` are superseded, and they read the same. `12f704f2` (2p) had 58.5 against 59.5. `1b01f4ac` (4p, both options, after the crash fix) had 25.0 against 28.5: −3.5 pp, 7 against 14, p = 0.19, and only 6 of its 200 games changed in the rerun.

reading. both options are correct now and neutral in strength at this budget. what they change is the shape of the search (a reused root starts with 2–3 times the nodes; the discard chain is a lattice), not the decision at 1024 sims. the strength question is a follow-up row: equal share at half the sims (not run). in self-play they save nothing either (cpu measurements, Section 6.1.2): the tree survives 43% (2p) to 53% (4p) of moves, but `advance` resets the budget, and transpositions restart the new root's shared children, so they fight reuse. the deployed bot can take them or leave them (Section 13.2.1).

reproduce

```
catanrl run configs/experiments/mcts-reuse-2p.yaml
catanrl run configs/experiments/mcts-reuse-4p-both.yaml
catanrl run configs/experiments/mcts-reuse-4p.yaml
# pair a new row with its stored row on the shared games, e.g.
uv run python tools/search_probe.py --pair aca296e23ff93a67 72819d1a1a072278
```

5 hidden information

in catan only development cards and robber steals are hidden. this section asks whether a bot gains from knowing the hidden cards better: an exact tracker of the other hands, a network head that predicts the hidden features, and acting on that prediction instead of on sampled worlds. so far nothing here moves the policy. the exact tracker is right and free but no agent plays differently for it, the prediction head is a small regulariser, and in 2p one predicted row is worth sixteen sampled worlds.

levels. the encoder hides 15 features of every other seat below the level that shows them: 5 resource ratios, actual victory points, 5 dev-card counts and 4 playable flags. `private` zeroes all 15 and is the only fair level past two players. in 2p every resource movement is seen by both seats, so `public` (the opponent's exact resource hand) is fair too. `full` also shows the opponent's dev cards and is fair only through `BELIEF` or a prediction (Section 4).

5.1 the hand tracker

5.1.1 an exact-posterior hand tracker DONE

ran 2026-10-17 · **idea** marco

> **tl;dr.** marco's two robbery examples and his 14th-knight example are now engine tests, and the existing pool tracker already resolves all three. a new exact tracker (`tracker=exact`) matches brute-force enumeration exactly, where the pool tracker's deal is off by 0.12–0.18 total variation on average. sampling costs the same.

question. marco's idea: the best possible tracker for any number of players, one that resolves hidden information as soon as later actions reveal it. his examples:

- we are A. B holds a wood and a sheep; C holds a brick, an ore and a wheat. B robs C, then buys a road: B now holds a sheep, C an ore and a wheat. if instead C robbed B and bought a dev card, C holds a brick and B a wood.
- we have played 6 knights, B 6 and C 1. when C plays another, that was the 14th and last knight: no hand and no deck holds one, so contending for largest army against B is pointless. more generally, every dev-card play should update the prior that sampled worlds are drawn from.

setup. the pool tracker (C++ `HandTracker`) keeps, per seat, a known lower bound per resource plus a count of unknown cards, and one global pool of unknown cards derived from the bank. that already resolves both robbery examples. the knight example is covered on the sampling side (`unseen_development_cards` is the deck minus my hand minus every played card), and the row carries every seat's played cards, so the network can count too. what the pool tracker lacks is a prior over which card a hidden steal took: a steal from brick, brick, ore is sampled 1:1 instead of 2:1.

the exact mode keeps weighted hypotheses over the hidden steals, branched by the victim's count of each kind, pruned by every later payment and merged when they reach the same hands. it is capped at 4096 hypotheses (the most seen in 424 simulated 2–6p games is 748, the average 2) and falls back to the pool past the cap. neither mode uses evidence from what seats chose not to do. `pool` stays the default.

result. on 160 positions with at least one hidden steal, against the true posterior by brute-force enumeration:

Table 16: total variation distance between each tracker's deal of the other seats' resource hands and the enumerated posterior.

tracker	mean TV from the truth	worst position	cost per sampled world
exact	0.0	0.0	about 12 µs
pool	0.12 (random play) to 0.18 (value play)	up to 0.9	about 12 µs

reading. the pool tracker's approximation is real, not just theoretical, and the exact tracker removes it for free. whether any agent plays better for it is the next two experiments.

reproduce

```
# the arguments of the 160-position run are not recorded; defaults otherwise
uv run python tools/belief_exactness.py --play random
uv run python tools/belief_exactness.py --play value
```

5.1.2 exact vs pool through BELIEF at 4p (void: a seating artifact) DONE

ran 2026-10-18 14:53–14:57 · **config** `configs/experiments/tracker-modes.yaml` · **wandb** tracker-modes, eval only, ppo-4p/main (no local mirror) · **records** `79147906`, `b07e5bd9`

> **tl;dr.** the first 4p table gave exact 40.4 against pool 52.7, but the gap is the seating, not the tracker. at the `private` level every sampled world encodes to the true row, so `BELIEF` with either tracker is the plain argmax policy, game for game. slot 0 always moves right after an F, and F robs the seat after it; every 4p head to head since plays both slot orders.

question. does `ppo-4p`'s belief policy play better with the exact tracker?

setup. `BELIEF:ckpt=ppo-4p/main@e3650:worlds=16` with `tracker=exact` in slot 0 and `tracker=pool` in slot 1, two F in slots 2–3, 500 boards × 4 rotations of `heldout-4p` (2,000 games, 3 min). every game is a paired comparison.

result.

Table 17: `ppo-4p/main@e3650` at one table with two F on `heldout-4p`. paired difference exact − pool −12.3 pp [−16.5, −8.1], the same at every seat.

lineup	slot 0	slot 1	record
`BELIEF` exact, `BELIEF` pool, F, F	40.4 38.2–42.5	52.7 50.5–54.8	`79147906`
`POLICY`, `POLICY`, F, F (`ppo-4p`'s own control)	40.45	52.75	`b07e5bd9`

a cpu replay of 1,208 decisions found that 0 of 4,832 worlds per mode differ from the true row in any feature: the encoder zeroes exactly what a world could change. so the record is game for game identical to the control. the gap is the lineup. the suite seats `lineup[(seat + r) mod 4]`, so slot 0 always moves right after the F in slot 3, and F (`enemies=next`) robs and blocks the seat after it. in the replayed games the F in slot 3 put the robber on slot 0's tiles in 280 of 280 moves; slot 0 was stolen from by an F 263 times, slot 1 39 times. with `F:enemies=all` the difference flips to about +7 pp, so even then the two learner slots are not exchangeable. what survived: the exact deal reproduces the other seats' true resource hands in 72.0% of worlds, the pool deal in 60.5% (no fall-back, 1–71 hypotheses), and the exact-mode env is bit-identical to the pool-mode env.

reading. a paired 4p table needs both slot orders, or the sum of the two learner slots. this voided the first 4p head to head of `ppo-mixed` against `ppo-4p` (Section 10). the tracker can only matter to an agent that reads the sampled hands: in 4p that is `MCTS` with sampled worlds, or a `full`-level 4p net played through `BELIEF` (none trained). in 2p it never matters: there are no hidden steals, so the two trackers coincide.

```
reproduce
catanrl run configs/experiments/tracker-modes.yaml
# the control is a ppo-4p final:
catanrl eval "POLICY:ckpt=ppo-4p/main@e3650" "POLICY:ckpt=ppo-4p/main@e3650" F F \
  --suite heldout-4p
```

5.1.3 exact vs pool inside 4p search DONE

ran 2026-10-19 07:11–07:38 · **config** `configs/experiments/tracker-search.yaml` · **wandb** tracker-search, eval only · **records** `89915b43`, `139796c6`

> **tl;dr.** inside a search that does read the sampled hands (256 sims, 4 worlds), exact and pool tie exactly: 193 wins each out of 400 over both slot orders. the two games of each pair are mostly the same game, so at this budget the better deal rarely changes a move. `tracker=exact` stays as a default-off option.

question. does the exact deal help an agent that plays out steals and builds with the sampled hands?

setup. `MCTS:sims=256:dets=4` with `ppo-4p/main@e3650`, exact against pool at one table with two F, 50 boards × 4 rotations of `heldout-4p` in each slot order (400 games). it took 27 min, not the estimated 3 h, because the searchers batch their leaves across the 256 games in flight.

result.

Table 18: MCTS 256 sims, 4 worlds, exact against pool with two F on `heldout-4p`. paired over both orders 0.0 pp [−3.2, 3.2] (bootstrap over the 200 board × rotation pairs).

order	slot 0	slot 1	record
exact first	exact 44.5 37.8–51.4	pool 52.5 45.6–59.3	`89915b43`
pool first	pool 44.0 37.3–50.9	exact 52.0 45.1–58.8	`139796c6`

the slot after the F loses 8 points in both orders (44.3 against 52.3): the seating artifact again, now on the searcher. on the same dice the same seat won 186 of the 200 pairs, in the same number of turns in 169, with the same number of decisions per seat in 161. the 14 pairs that changed hands split 6 to pool, 4 to exact, and 4 where exact won one game and an F the other.

reading. the exact posterior is right (identical to enumeration, where the pool deal is off by 0.12–0.18 total variation) and costs nothing, but no agent in this repo plays differently for it. the `private`-level policies cannot see it, and search at the sim counts we can afford does not act on it. what would read it is open (Section 5.3.1).

reproduce

```
catanrl run configs/experiments/tracker-search.yaml
```

5.2 predicting the hidden features

5.2.1 a head that predicts the hidden features, 2p DONE

ran 2026-10-18 13:11–14:53 · **idea** marco · **config** `configs/experiments/aux-hidden.yaml` · **wandb** aux-hidden/main · **records** `0e28d5cf`, `c6b7f682`

tl;dr. marco's idea: train the model to predict the full-information observation from what it sees. an auxiliary head that regresses the opponent's 15 hidden features does not improve the 2p policy: 89.1 against F versus 90.7 without it, and 47.7 head to head. what it does show is that in 2p the opponent's resources are almost fully inferable, and the dev cards are not.

question. does a trunk that has to encode the hidden features play better?

setup. the target already exists: the `full` row of the same position. the head regresses the 15 hidden features of each other seat (a test pins the set against the encoder) from the `full` players block the row carries for a critic at another level (`critic_level: full`). the trunk never reads that block: a test perturbs it and checks that logits, values and predictions are bit-identical, so the policy stays fair and evals encode only the `private` row. the loss is a masked MSE over present opponent seats on acting rows, with coefficient 0.3 from the measured gradient ratio against ppo's loss; at 0 the training is bitwise the plain recipe. the run is the arch ablation's `graph-s` cell plus the head (100M steps, 2p, `private`), judged against F and head to head against that cell on `heldout-2p`, 5,000 games a row.

result.

Table 19: the aux cell against the plain `graph-s` cell, same recipe and budget, on `heldout-2p`.

cell	params	steps/s	select-2p (best)	vs F	vs `graph-s`
`graph-s` (Section 2)	0.54M	17.6k	92.5 (e750)	90.7 89.9–91.5 `039b60f0`	—
`graph-s` + aux head	0.62M	17.1k	90.0 (e700)	89.1 88.2–89.9 `0e28d5cf`	47.7 46.3–49.1 `c6b7f682`

the aux cell took 1.7 h, and its critic (explained variance 0.930 against 0.928) and entropy ended where the plain cell's did. per-feature R^2 of the head over the last 50 epochs (from the logged `aux/mse_*` and `aux/var_*`): resources 0.90–0.99, actual vp 0.99, vp cards 0.85, knights 0.60, monopoly, road building and year of plenty 0.10–0.22, the playable flags the same. the per-feature table, next to 4p, is in Table 21.

reading. at coefficient 0.3 the head acted as a regulariser that cost about a point and a half and nothing else. in 2p the tracker's known cards, the hand size and the production history leave little to infer, so the trunk had nothing new to encode. the dev cards are the genuinely hidden part and stay hidden, except vp cards: a card bought and never played is a vp card, and the net works that out.

reproduce

```
catanrl run configs/experiments/aux-hidden.yaml
# the head-to-head final needs the plain cell:
catanrl run configs/experiments/arch-ablation.yaml --only graph-s
```

5.2.2 the same head at 4p DONE

ran 2026-10-19 10:15–14:02 · **config** `configs/experiments/aux-hidden-4p.yaml` · **wandb** aux-hidden-4p/plain, aux-hidden-4p/aux · **records** `d97f69b3`, `da4cb5be`, `27a774b6`, `2eb6a290`

> **tl;dr.** at 4p the other hands are genuinely hidden: resource R^2 falls to 0.48–0.59 from 0.90–0.99 in 2p, and the head recovers about half their variance. the policy still gains nothing: 64.0 against 63.2 versus three F, and 42.5 against 42.5 head to head over both slot orders.

question. steals and the robber hide more at 4p. does the head help where there is more to infer?

setup. `ppo-4p`'s recipe (`private` level, the lambda critic default) at 750 epochs: 196.6M agent steps = 49.2M decisions, the 2p cells' budget per decision. two arms, plain and aux (coefficient 0.3, the critic at the `full` level as in 2p): 1.85 h and 1.92 h at 30.5k and 29.6k steps/s. select-4p every 50 epochs (800 games). finals: each arm against three F, and the two arms at one table with two F in both slot orders (8,000 games).

result.

Table 20: `aux-hidden-4p` on `heldout-4p`, shares, parity 25. vs three F is 4,000 games per arm. head to head is the two arms at one table with two F, in both slot orders (8,000 games, `27a774b6` and `2eb6a290`): paired 0.0 pp [−2.0, 1.9].

arm	select-4p (best)	vs three F	head to head	record
plain	65.8 (e600)	**64.0** 62.5–65.5	42.5	`d97f69b3`
aux	68.5 (e700)	63.2 61.6–64.6	42.5	`da4cb5be`

Table 21: R^2 of the aux head per hidden feature over the last 50 epochs. what the public row already determines (2p resources, actual vp) the head gets nearly right; dev cards stay hidden at both player counts.

hidden feature	2p (`aux-hidden`)	4p (`aux-hidden-4p/aux`)
wheat	0.99	0.59
ore	0.96	0.57
sheep	0.96	0.52
brick	0.93	0.48
wood	0.90	0.55
actual victory points	0.99	0.95
vp cards in hand	0.85	0.65
knights in hand	0.60	0.51
other dev cards, playable flags	0.10–0.22	0.07–0.21

at 4p the knight R^2 was 0.70 at epoch 100 and fell to 0.53 by the end. the selects overstate the finals by 2–5 points (best of sixteen noisy selects). the slot after the F loses 9 points in both orders (38.0 against 47.0), the seating artifact of Section 5.1.2.

reading. steals and the robber hide half the variance of the other hands from the public row, the head recovers that half, and the policy still gains nothing from a trunk that encodes it. the head is a regulariser at both player counts; what paid was using the prediction in place of sampled worlds (Section 5.2.3). the plain arm doubles as the lambda critic's from-scratch test (Section 3) and the markov net's control (Section 12), where the same head also probes the lstm's memory.

reproduce

```
# plain first: the aux arm's head-to-head finals need it
catanrl run configs/experiments/aux-hidden-4p.yaml
```

5.2.3 `PREDICT`: one predicted row instead of sixteen sampled worlds DONE

ran 2026-10-18 17:45–18:03 · **idea** marco · **config** `configs/experiments/predict-vs-belief.yaml` · **wandb** predict-vs-belief, eval only · **records** `b74d240b`, `e7e34d28`, `a939fa5f`, `751509ea`

> **tl;dr.** the full-information network, acting on its `private` row with the hidden features filled in by the aux head's prediction, plays as well as the same network averaged over sixteen sampled worlds: 50.4 head to head and 94.7 against F, with two forward passes instead of sixteen. in 2p one predicted row is worth sixteen sampled worlds.

question. the literal reading of marco's idea: act on the prediction instead of sampling worlds.

setup. `PREDICT:ckpt=puffer-graph2/full@e3350:predictor=aux-hidden/main@e700` overwrites the opponent's 15 hidden features in the true game's `private` row with the aux head's prediction and plays the `full` network on it. it

is fair by construction, and tests pin it: the `private` row plus the fill is bitwise the `full` row, and with the true features in the head's place the agent reproduces the `full` policy's actions exactly. `heldout-2p`, 5,000 games a row, 18 min for all four rows.

result.

Table 22: `PREDICT` with `puffer-graph2/full@e3350` and the `aux-hidden/main@e700` predictor on `heldout-2p`, 5,000 games a row.

row	score	record
`PREDICT` vs F	**94.7** 94.1–95.3	`b74d240b`
`PREDICT` vs `BELIEF` 16 worlds, same `full` checkpoint	**50.4** 49.1–51.8	`e7e34d28`
`PREDICT`, predictions snapped to the encoder's grid, vs `BELIEF` 16 worlds	49.9 48.5–51.3	`a939fa5f`
`PREDICT` vs `arch-ablation/graph-s` (`private`, 100M steps)	63.1 61.8–64.4	`751509ea`
reference: `BELIEF` 16 worlds vs F (Section 3)	95.06 94.42–95.63	`67869e7a`
reference: `full` policy on the true row vs F, unfair	95.16 94.53–95.72	`ff8528a4`

reading. this fits the R^2 table: what the predictor gets right (resources, actual vp) is what the tracker's worlds agree on anyway, and what it cannot know (dev cards) averages out either way. the last row is confounded by budget (100M steps against `puffer-graph2`'s 500M) and says nothing about the fill by itself.

```
reproduce
catanrl run configs/experiments/predict-vs-belief.yaml
```

5.3 open

5.3.1 what would read a better belief at 4p TODO

idea assistant

> **tl;dr.** no agent in the repo plays differently for the exact tracker or for a 4p predictor. a `full`-level 4p net played through `BELIEF`, or search with many more worlds, would. none is queued.

plan. (my suggestion, not queued) train a 4p net at the `full` level and play it through `BELIEF`, as 2p does; whether 4p should train at `full` is itself an open question. that net plus a 4p predictor is also what `PREDICT` needs at 4p. a predictor head on the `full`-level network's own trunk would make `PREDICT` one network again. search with many more than 4 worlds per decision is the other agent that would read the exact deal.

6 alphazero

marco's note on the idea: "currently agreement is really high, doesn't seem to yield many gains". the question is whether training the network on its own search's choices (alphazero self-play) gives a better network, as a raw policy or inside the search. the loop (`train.algo: alphazero`) self-plays through the batched search with gumbel roots, keeps a replay buffer of (position, visit-count policy, per-seat outcome) and warm-starts from a ppo checkpoint. so far no run beat its start: the raw policy got worse every time, inside the search the trained networks played the same (e8) or worse (the soft run), and the likeliest culprit is how the targets are shaped.

6.1 first runs and cost

6.1.1 30-minute sanity run from a weak ppo net DONE

ran 2026-10-14 · **idea** marco · **config** `configs/experiments/az-from-ppo.yaml`

> **tl;dr.** a first end-to-end test of the loop, from a weak 40-minute ppo net at 57.8% vs F. the raw policy got steadily worse, down to 18.5% vs F, while fitting noisy 64-sim targets. it says nothing about strong networks or bigger searches.

question. does the loop run, and does a weak network learn from its own 64-sim search?

setup. the az stage of `az-from-ppo`, run for 30 minutes in a scratch runs dir (since deleted, so no records and no wandb mirror). the start was a 40-minute ppo sanity net, not puffer-graph2. oracle search (`dets: 0`: the search sees the true hidden state), gumbel targets at 64 sims on a quarter of moves and 16 on the rest, pure self-play, lr 5e-5, 25 iterations. nothing was played head to head against the start.

result.

Table 23: `az-from-ppo`'s az stage. no records survive, so there are no intervals.

player	vs F	games
start, raw policy	57.8	
start, 64-sim search	63.5	100
trained raw policy, three successive checkpoints	46.6 → 24.4 → 18.5	600 each

reading. the targets were noisy. with q min-max rescaled over the root's moves, a near-single-visit q gap swings a target by ~6.5 logits. two search seeds agree on the top move 87% of the time (60% on discards). knight-before-roll (how often the policy plays a knight before rolling) went from 0.90 to 0.27, and fitting the noise took the network's entropy from 0.42 to 0.82 until its argmax drifted to end-turn. the run found one small bug, the gumbel temperature-0 finalist (fixed in cppanatron `5d9dda9`), which doesn't touch the targets. for context, v1 never trained above 512 sims, and 512-sim targets didn't move an 87% network (v1's EX-1d, a frozen-teacher distillation of an 8 × 64-sim search: +1.3 vs F and 50.6% head to head, both null).

reproduce

```
# runs the ppo stage (50M steps) and then 300 az iterations; the original ran
# only the az stage, for 25 iterations, from a 40-minute ppo net
catanrl run configs/experiments/az-from-ppo.yaml
```

6.1.2 self-play cost study (cpu) DONE

ran 2026-10-21 · **idea** assistant

tl;dr. before spending gpu time i measured on cpu what makes self-play expensive. tree reuse and transpositions save nothing as built; an eval cache across one self-play step would cut network forwards by 31–46%. a config-only first run at 1024-sim targets costs ~13–22 gpu hours for 60 iterations.

question. does the search reuse built for the deployed bot make self-play cheaper, and what does a first run cost?

setup. cpu measurements of self-play searches in 2p and 4p. the 2p numbers were measured under the engine's default rules (10 vp, discard above 7), not base-2p's (15 vp, discard above 9); the target probes of `az-2p-2048` (Section 6.2.1) used base-2p's rules.

result.

Table 24: cpu measurements of self-play search.

lever	effect on self-play
tree reuse, as built	the tree survives 43% (2p) to 53% (4p) of moves, but `advance` resets the budget, so it saves no leaves
engine top-up budget (search only sims − kept visits)	15–20% fewer leaves. kept visits are worth about their count, so reuse can't halve the budget
transpositions	nothing. they fight reuse (they restart the new root's shared children), so they stay off
eval cache across one self-play step	31–46% fewer forwards. hashing has to move into the engine

a 1024-sim move costs ~1,000 leaf rows and ~80 ms of engine cpu. a first run as a config-only change of the existing trainer (1024-sim gumbel targets on every move, 512 games, from `puffer-graph2/public`) is ~13–22 gpu hours for 60 iterations.

reading. the real lever is the eval cache, and it needs engine work. marco chose to go straight to the loop at 2048 sims and do engine work only after a positive result (Section 6.2.1). the engine levers are listed in Section 6.4.2.

reproduce

```
# not reproducible: one-off cpu measurements, no script for them is in the repo
```

6.2 the 2p specialist at 2048-sim targets

6.2.1 az-2p-2048: self-play from the 2p specialist STOPPED

ran 2026-10-21 15:40–18:53 · **idea** marco · **config** `configs/experiments/az-2p-2048.yaml` · **wandb** az-2p-2048/main · **records** `e34b2057`, `fa218f06`, `14f41aa1`

> **tl;dr.** marco's pick: the best 2p network straight into self-play, with 2048-sim targets on every move. its raw policy lost to its start (48.0 and 48.2 head to head), so the stop rule ended the run after 8 iterations. the cause looks like the targets: gumbel's target puts ≥ 0.99 on one move in most decisions, whatever the real gap between moves.

question. does training on 2048-sim search targets improve an already strong network (95% vs F)?

setup. start `puffer-graph2/public@e3600` (actor and critic), base-2p, 512 games in flight. fair gumbel search on every move (`dets: 1`, one world dealt from the tracker per search; m = 16 root moves considered). 30 iterations × 16,384 targets planned, lr 5e-5. every 2 iterations the raw policy plays its start head to head on `select-2p-2k` (4,000 games); the search finals run only if that is positive. logging-only monitors (training is bit-identical with or without them): the value head's loss on finished games before it trains on them, how often the search overrules the network's argmax, and drift from the start on a fixed probe of 4,096 positions. stop rule: two heads to head ≤ 48.5 in a row.

cpu probes under base-2p rules came first. 2048-sim targets cut the prior's distance to an 8192-sim reference from 0.295 to 0.197 tv (the reference's own noise floor is 0.179) and change the top move at 20% of positions. they are barely better than 1024 sims (−0.016 −0.034–0.001). robber moves, and positions where the opponent holds a hidden vp card, stay noisy. the first iteration ran at 23.6k leaves/s and took 24 min, so ~12 h for the 30.

result. stopped at e8 (3.2 h). best stayed e0, so no finals ran (`az-2p-2048-search.yaml` was never played).

Table 25: `az-2p-2048` selects on `select-2p-2k`. e2 and e4 equal e0: the first update came in iteration 5, once enough games had finished.

raw policy vs its start	score	games	record
e0 (the same weights)	50.0 48.5–51.5	4,000	`e34b2057`
e6	**48.0** 46.5–49.5	4,000	`fa218f06`
e8	**48.2** 46.7–49.7	4,000	`14f41aa1`

from the start to e8 the network's entropy rose from 0.088 to 0.125. the search overruled its argmax more often (0.18 ⟶ 0.23), where a policy that learns from its search should make that rarer. knight-before-roll on the drift probe fell from 0.53 to 0.43.

reading. it went the way the sanity run went, only slower. the critic wasn't the cause. the monitor's out-of-sample value loss rose from 0.20 to 0.54, but that is the games getting longer (the first games to finish are the short ones). on the same rows the start scores 0.375 / 0.508 / 0.387 at iterations 6–8 and the trained net 0.359 / 0.538 / 0.402, a few hundredths either way.

the targets were the cause, as far as one look can say. gumbel's $\sigma(q) = (50 + \max \text{visits}) \cdot 0.1 \cdot q$, with q min-max rescaled to [0, 1] across the root's moves, puts a ~100-logit spread between the best and the worst move at 2048 sims, whatever the gap in win probability. so all 16,153 two-move decisions in the replay (mostly knight or roll) got a target ≥ 0.99 on one move, and so did 81% of the decisions with more than eight legal moves. on knight-or-roll the start was ≥ 90% sure 98% of the time. the targets went against its argmax 21% of the time and played the knight 44% of the time against the start's 53%: the drift the network then learned.

the moves behind the targets are good: the same search over the start's weights beats the raw policy 62.4 (`831eea84`, Section 6.2.2), far above puct over the same policy (54.9 at 1024 sims, 55.2 at 4096, Section 4). the search had plenty to teach, and the raw policy still got worse learning it.

reproduce

```
catanrl run configs/experiments/az-2p-2048.yaml
```

6.2.2 e8 judged inside the search DONE

ran 2026-10-21 18:56 – 2026-10-22 02:59 · **idea** marco · **config** `configs/experiments/gumbel-edge-2p-e8.yaml` · **wandb** gumbel-edge-2p, eval only, gumbel-edge-2p-e8, eval only · **records** `831eea84`, `c4ea7b52`

> **tl;dr.** marco's point: the goal is a better network inside the search, so judge alphazero there, not by the raw policy. in the same 2048-sim search against the same raw start, e8 scores 61.8 and the start's own weights 62.4: −0.6 pp paired, no difference. the training didn't make a better search network either.

question. does e8 search better than its start, even though its raw policy got worse?

setup. two rows, each the same 2048-sim gumbel search (`dets: 1`, m = 16, `c_puct` 0.75) against the same raw start (`puffer-graph2/public@e3600`), 1,000 games on the first 500 boards of `heldout-2p` in both seat orders. only the network inside the search differs. the rows are paired: they play the same game ids, so they are compared game by game, with mcnemar's test on the games only one of them won (`tools/search_probe.py --pair`).

result.

Table 26: 2048-sim gumbel search vs the raw start on `heldout-2p` (500 boards, both seat orders).

network inside the search, vs the raw start	score	record
start (e0)	62.4 59.4–65.3	`831eea84`
e8 of `az-2p-2048`	61.8 58.7–64.8	`c4ea7b52`
e8 − e0, paired over the 1,000 shared games	**−0.6 pp** −3.7–2.5	124 games won only by e8, 130 only by e0, mcnemar p = 0.75

reading. the raw policy lost two points and the search played the same, so stopping at e8 cost nothing, but by luck. only a quarter of the pairs differ at all (the board, the seat and the dice decide most games), so pairing roughly halves the interval two independent rows would give. this paired test is the template: from here on every alphazero run is judged this way, and so is the critic refit (Section 7).

reproduce

```
# the yaml of the e8 row names runs/az-2p-2048/main/checkpoints/e000008.pt by absolute path
catanrl run configs/experiments/gumbel-edge-2p.yaml      # start's weights in the search: 831eea84
catanrl run configs/experiments/gumbel-edge-2p-e8.yaml   # e8 in the search: c4ea7b52
uv run python tools/search_probe.py --pair c4ea7b5234eb6a50 831eea84f04c89d8
```

6.3 softer targets

6.3.1 az-2p-2048-soft: softer targets through a tempered prior STOPPED

ran 2026-10-22 15:49 – 2026-10-23 01:57 · **idea** marco · **config** `configs/experiments/az-2p-2048-soft.yaml` · **wandb** az-2p-2048-soft/main · **records** `856a0421`, `ae5a4000`, `f016fee8`, `8906dcf7`

> **tl;dr.** marco's next step: the same loop with the search unchanged and softer targets. the softening that looked best on cpu, tempering the prior inside the target, flattened the network instead: its search fell from 57.4 to 46.7 against the raw start by e15, and marco stopped the run. the prior temperature (my proposal) is the likely culprit, since canopy never tempers the policy its search reads.

question. does a network trained on softer versions of the same search's targets search better?

setup. the trainer rebuilds gumbel's improved policy itself, from each search's visits and q and the network's root outputs (`train.policy_target: gumbel`). at the engine's constants it matches the engine's target to ~3e-8, and the default trains bit-identically to before.

the softening was picked on cpu: 600 positions of the start's raw-policy base-2p self-play, two 2048-sim searches each, scored by −log of the target's mass on an 8192-sim search's move. a gentler σ alone softens nothing. the start's prior is itself one-hot (a median gap of 47 nats between its top two logits on knight-or-roll decisions), so softmax(prior + σ) stays one-hot at any scale. what does soften it is tempering the prior inside the target: softmax(logits / 10 + (50 + max visits) · 0.15 · q), with q unrescaled (my proposal from the probe, approved by marco).

Table 27: cpu probe, 600 positions. the score is –log of the target's mass on the 8192-sim move, relative to the engine's target; lower is better.

target	score vs engine (nats)	agrees with the 8192 move	entropy	≥ 0.99 on one move
engine (σ on rescaled q, `c_scale` 0.1)		81.4%	0.05	87%
best σ-only setting	−0.10 −0.24–0.03			
tempered prior (logits / 10, `c_scale` 0.15, q unrescaled)	**−1.12** −1.39–−0.86	80.5%	0.64	22%

that is calibration, not better moves: two 8192-sim searches pick the same move only 86% of the time, and the expected regret under the softer target is slightly worse (+0.05 0.01–0.08 win points).

everything else is as in `az-2p-2048`. it is judged in the search throughout, as marco asked: every 5th iteration the checkpoint in a 512-sim gumbel search plays the raw start on the same 1,000 games of `select-2p-500`, paired with e0's row. the planned finals put the best and the last checkpoint in the 2048-sim search against the raw start on the games of `831eea84`, each paired with it (mcnemar, holm over the two). every iteration's weights are kept, so an e8 row can pair with `c4ea7b52` later. stop rule: only a collapse, two selects in a row more than 4 pp below e0 on the same games (about 2.5 paired sds); the raw policy is not a stop criterion this time. planned cost: ~19 h of training and selects, then 4–8 h of finals.

result.

Table 28: `az-2p-2048-soft` selects: 512-sim gumbel search vs the raw start, 1,000 games of `select-2p-500` each.

checkpoint in the search	score vs raw start	paired vs e0	only e0 / only eN	mcnemar p	record
e0	57.4 54.3–60.4				`856a0421`
e5	56.0 52.9–59.0	−1.4 pp	129 / 115	0.41	`ae5a4000`
e10	53.8 50.7–56.9	−3.6 pp	158 / 122	0.036	`f016fee8`
e15	**46.7** 43.6–49.8	**−10.7 pp**	225 / 118	$< 10^{-6}$	`8906dcf7`

the monitors say why. by e13 the network's entropy was 0.68 (0.09 at the start), it agreed with the start's argmax on 72% of the drift probe, and the search overruled its argmax 41% of the time (18% at the start). at e15 the entropy was 0.92 and the search's depth had fallen from 11 to 8.2. the collapse rule would have needed e20 too; marco stopped the run after e15 ("it is sucking balls"). best stays e0, so there are no finals.

reading. the softer target pulled the prior flat faster than the search's q could replace what the prior knew. the canopy comparison (Section 6.4.1) puts the blame on the prior temperature: canopy never tempers the policy the search reads.

reproduce

```
catanrl run configs/experiments/az-2p-2048-soft.yaml   # marco stopped it by hand after the e15 select
uv run python tools/search_probe.py --pair 8906dcf7adfc1c96 856a0421185ce86f   # e15 vs e0, paired
```

6.4 what next

6.4.1 what canopy did, and where we diverge DONE

ran 2026-10-23 · **idea** marco

tl;dr. marco asked how canopy trained its alphazero network. canopy trained from scratch with a recipe close to ours, but published no win rates, and nobody has shown the recipe improves an already strong network. the likeliest reasons our runs failed are our target's q scaling and the soft run's prior temperature.

question. what did canopy do, and which of our differences could explain our two failed runs?

setup. read from cullback/canopy's code and history, checked against source, plus one more measurement on the 600-position cpu probe of Section 6.3.1.

result. canopy's recipe next to ours:

Table 29: canopy's alphazero recipe against ours.

	canopy	**ours (`az-2p-2048`)**
start	a 2.4M-parameter nexus-v3 net, most likely from random weights	`puffer-graph2/public@e3600`, 95% vs F
rules	1v1 colonist (balanced dice, friendly robber)	base-2p
search	gumbel, m = 6; 1600 sims on a quarter of moves, 64-sim value-only searches on the rest; an improved-policy interior	gumbel, m = 16; 2048 sims on every move
hidden cards	the opponent's dev cards resampled every simulation	one tracker world per search
q in the target	divided by the span of values the search saw, `c_scale` 1	min-max rescaled over the root's moves, `c_scale` 0.1
softening	a soft-policy side output (π^(1/4), weight 8) that the search never reads	none (`az-2p-2048`); a tempered prior inside the target (`az-2p-2048-soft`)
value	win/draw/loss, blended from the outcome toward the search's q (to 85% over 60 iterations)	per-seat outcome
replay, optimizer	500k positions; adamw 5e-4 ⟶ 1e-4	400k positions; lr 5e-5, a fresh optimizer on a muon-trained net
length	about 315 iterations (~47M positions, inferred from the checkpoint name)	8 and 15 iterations

canopy's evidence of success is thin. it publishes no win rates, elo or opponents, only a readme line ("we believe … the strongest public 1v1 catan agent"). the only outside numbers are v1's: its graph checkpoints scored 23–33% against canopy at 64 sims (48 games each).

the divergences most likely to matter:

1. **how the target scales q.** our engine min-max rescales q over the root's moves at `c_scale` 0.1. so on a two-move decision any q lead becomes a ~107-logit lead. canopy divides by the span of values its search saw, at `c_scale` 1, so near-ties stay near-ties. on the probe's 600 positions a canopy-shaped target (unrescaled q, `c_scale` 1, no temperature) overrules the start on knight-or-roll 2.2% of the time, against the engine target's 18.9%. it agrees with the 8192-sim move as often (88.1 vs 87.8%) and stays sharp (entropy 0.08). the –log score of Section 6.3.1 couldn't see this difference.
2. **the prior temperature.** the soft run's temperature is not canopy's (canopy softens only through its separate output), and it is what flattened the soft run.
3. **the start.** canopy's search targets were always far better than its prior. from a 95% start the search's gain is close to its own noise. the hand-off also switches the critic's objective and starts a fresh adamw on a muon-trained net.
4. **hidden cards.** one sampled world per search, against canopy's per-simulation resampling.

reading. v1 ran canopy's recipe (its puct version) on a 55%-vs-F graph net and it did improve it, 55 ⟶ 70.5% raw vs F over 10 iterations, but never from a ≥ 90% start. divergences 1 and 4 are the candidate next runs (Section 6.4.2).

reproduce

```
# not reproducible: a code reading of cullback/canopy plus the cpu probe, whose script is not in the repo
```

6.4.2 follow-ups not run TODO

idea assistant

tl;dr. candidate next steps from the analyses above, all my suggestions. marco moved all alphazero work to the bottom of the queue on 2026-10-24: canopy loses to our ppo-plus-search bot, so ppo into search is the path.

plan.

- a canopy-shaped target: q unrescaled, `c_scale` 1, no prior temperature (divergence 1 of Section 6.4.1). (my suggestion, not queued)
- per-simulation world resampling instead of one world per search (divergence 4). (my suggestion, not queued)
- an offline distillation gate before the online loop: one fixed set of 1024-sim targets, several policy-only fits judged head to head. it would answer "is there anything to distill" more cheaply than the online loop. (my suggestion, not queued)

- cheaper self-play: an engine top-up budget (15–20% fewer leaves) and an engine-side eval cache (31–46% fewer forwards), from Section 6.1.2. (my suggestion, not queued)
- the warm-start critic is a discounted v-trace critic while alphazero trains undiscounted win/loss, so the value head drifts during the test. the critic refit (Section 7) measures that discount.

the search finals of `az-2p-2048` (`configs/experiments/az-2p-2048-search.yaml`: best and init in the 2048-sim search vs the raw start) wait on a positive raw result and were never played.

7 refitting the critic after ppo

marco's idea: during multi-agent ppo the critic chases a policy that keeps changing, so it probably never converges on the final one. freeze the policy, keep training the critic on many of the final policy's games, and mcts should get better with the more accurate critic. a second reason to expect a gain: the start's critic is discounted (v-trace targets at $\gamma = 0.999$, $\lambda = 0.95$). so it predicts something closer to 0.999^(moves left) × p(win) than p(win). that is also not the undiscounted win/loss alphazero's value head trains on. result: the refit calibrated the critic (it no longer discounts) but did not make it rank positions better, and the search did not get stronger: +2.1 pp at 512 sims ($p = 0.15$) and +0.1 pp at 2048 sims ($p = 1.0$). the idea fails its pre-registered rule, so the value-head-only arm was not run.

7.1 the start's critic

7.1.1 cpu probe of the start's critic DONE

ran 2026-10-23

> **tl;dr.** before the gpu run i scored the start's critic against real outcomes on cpu. it is discounted as expected (the two seats' values sum to 0.936) but ranks positions well. the only headroom is calibration, about 0.01 nats, which the gumbel root mostly cancels, so on this evidence alone i don't expect the refit to move the search rows.

question. the first step in the plan for marco's refit: how far is the start's critic from the undiscounted outcome, and how much can a refit gain on held-out games?

setup. 5,120 raw-policy games of `puffer-graph2/public@e3600` on its own training boards (827k decisions). every value is scored against the game's undiscounted outcome.

result.

Table 30: the start's critic (`puffer-graph2/public@e3600`) on 5,120 raw-policy games.

measure	value
log loss vs the undiscounted outcome	0.370 (constant guess 0.682)
the same by game stage	0.68 in the first tenth of a game ⟶ 0.11 in the last
the two seats' values summed	0.936 (1 if undiscounted)
calibration	a predicted 0.74 wins 83% of the time
held-out gain: each seat's value ÷ the two seats' sum, no training	0.010 0.008–0.011 nats
held-out gain: refit of the value head on the frozen trunk (the best fit)	0.012 nats
auc within the same stage of the game	0.894 for all

reading. it is discounted as expected: v ≈ 0.999^(decisions left) × p(win) fits where v ≈ p(win) does not, so it is under-confident about wins. the only headroom is calibration, and neither fix ranks positions any better. the gumbel root rescales q to its own range, which cancels most of a calibration change. whether a separately trained critic trunk on ~400k games finds discrimination the value head couldn't is what the gpu run tests (Section 7.2.1).

reproduce

```
# not reproducible as run: a one-off cpu probe whose script is not in the repo.
# tools/critic_heldout.py, the gpu run's held-out scorer, computes the same measures
# (log loss against the outcome, auc within turn bins, the seat-sum fix) for any checkpoint.
```

7.2 the refit

7.2.1 critic-refit-2p: frozen policy, refit critic, judged in the search DONE

ran 2026-10-23 09:57 – 2026-10-24 10:45 (paused 13:40–00:54 for canopy) · **idea** marco · **config** `configs/experiments/critic-refit-2p.yaml` · **wandb** critic-refit-2p/main, az-2p-2048-soft (baseline 856a0421), gumbel-edge-2p, eval only (baseline 831eea84) · **records** `576848d8`, `f841ca86`, `9641a3aa`, `856a0421`, `831eea84`

> **tl;dr.** a separate copy of the critic, refit on ~400k of the frozen policy's games, is calibrated (the discount is gone) but ranks positions no better than the start's. inside the search it changes nothing: +2.1 pp at 512 sims (p = 0.15), +0.1 pp at 2048 (62.5 vs 62.4, p = 1.0). a more accurate critic is not what limits our search.

question. with only the critic different, does mcts with a refit critic beat the raw start more often than mcts with the start's own critic?

setup.

- **frozen policy.** `train.freeze: [actor]` takes the policy out of the optimizer. its logits stay bit-identical through training, checked at every launch against `init_from`.
- **split trunk.** `init_from.split_shared` copies the shared trunk into separate policy and critic networks that reproduce the start exactly: identical logits, values and 512-sim searches (merged `3dc0450`). the e0 select (`576848d8`) plays the same games as the start's `856a0421` (identical games hash).
- **arm `main`.** refits the whole split critic (376k parameters) on the monte-carlo outcome ($\gamma = 1$, $\lambda = 1$) of raw-policy self-play in the start's own env. adam 3e-5 ⟶ 3e-6 over 500 epochs, ~400k games of the frozen policy at temperature 1, of which ~63% reach the loss. 256-step segments, so about two thirds of the targets are real outcomes and the rest bootstrap. ~8–14 gpu hours, not yet measured with two trunks.
- **held-out scoring.** a fixed set of 2,048 held-out games on boards outside the training pool (`tools/critic_heldout.py`) scores every checkpoint, paired, against two calibration-only fixes of the start. the run counts as converged if e500 is within 0.002 nats of e450.
- **decision rule, fixed before launch.** e500 inside mcts against the raw start, in two rows, each paired with the start's weights in the same search on the same games (mcnemar, holm at 0.05 over the two). at ~1.6–1.8 pp paired se, 80% power needs an effect of ~5 pp, so a null reads "no effect of 5 pp or more".

Table 31: the two rows that decide `critic-refit-2p`, and the rows they pair with.

decision row (e500 in mcts vs the raw start)	baseline: start's weights	baseline record
512 sims, the 1,000 games of `select-2p-500`	57.4 54.3–60.4	`856a0421`
2048 sims, `heldout-2p` (500 boards, both seat orders)	62.4 59.4–65.3	`831eea84`

result. the main arm was paused at e266 for the canopy search row (Section 8; the two don't fit in 8 gb together) and resumed on 2026-10-24 00:54; training ended at 03:00, the 2048-sim row at 10:45.

- **held-out critic** (2,048 games on boards outside the training pool, 330k decisions, game-clustered cis, paired per decision): cross-entropy against the outcome went from 0.3634 at e0 to **0.3460** at e500, −0.0174 nats −0.0212–−0.0134. ranking within a turn did not move (auc 0.9010 ⟶ 0.9016). a calibration-only fix of the start, platt scaling by turn, reaches 0.3470, so the refit beats it by only −0.0011 −0.0023–+0.0002 normalizing the start's two seats, v/(v0+v1), already gets 0.3499. converged: e500 is within 0.0001 of e450 (the rule was 0.002), and the curve is flat from about e100.
- **512-sim search** (the 1,000 games of `select-2p-500`): e500 scores **59.5** 56.4–62.5 (`f841ca86`) against the start's 57.4 (`856a0421`) on the same games, paired +2.1 pp, 105 games won only by e500 against 84 only by the start, mcnemar p = 0.15. not significant at the holm threshold (0.025 for the smaller p).
- **2048-sim search** (`heldout-2p`, 500 boards, both seat orders): e500 scores **62.5** 59.5–65.5 (`9641a3aa`) against the start's 62.4 (`831eea84`) on the same games, paired +0.1 pp, 110 against 109, mcnemar p = 1.0.
- **decision:** neither row passes holm (smallest p 0.15 against 0.025), so main fails and the head arm is not run.

reading. the refit fixed what the probe said was wrong, the discount, and nothing else: the critic is now calibrated but ranks positions no better, and almost all of the calibration gain is available for free by rescaling the start's values. the search only compares positions and moves, so calibration alone can't help it, and at 2048 sims the two critics win and lose the same games (219 of 1,000 differ, split evenly). a better critic for the search would have to rank better, not just be calibrated. by e50 the critic's two seats' values summed to 1.000, against the start's 0.936.

reproduce

```
catanrl run configs/experiments/critic-refit-2p.yaml --only main   # resumes from e266 in the same runs dir
uv run python tools/critic_heldout.py play --run critic-refit-2p/main --every 25 \
    --games 2048 --device cuda --out heldout-main.npz
uv run python tools/critic_heldout.py report heldout-main.npz --json heldout-main.json
```

7.2.2 head-only arm, and main vs head PARKED

idea marco · **config** `configs/experiments/critic-refit-2p.yaml`

> **tl;dr.** arm `head` refits only the value head on the shared trunk (74k parameters). it was to run only if a main row passed. neither did, so it is parked.

plan. same config, same decision rows as Section 7.2.1, one trunk instead of two. launch with `catanrl run configs/experiments/critic-refit-2p.yaml --only head`, only after a positive main row.

8 benchmark against released canopy

marco asked for a benchmark of the released canopy against our 2p bot "before anything else". canopy is a rust alphazero that calls its catan bot the strongest public 1v1 agent, with no published win rates (Section 6). so far its raw network is far weaker than ours (30.5%), and its full 1600-sim search played our raw policy about even over 60 games, too few to say more. on canopy's gpu build its search scores 33.0% against our 2048-sim search over 1,000 games, so our search wins two games in three. the other three rows are held: marco is choosing between our current rules and openhex's 1v1 rules (friendly robber, canopy's own robber rule) against the retrained 1v1 bot.

Table 32: the canopy benchmark at a glance. rows keep the numbers of the original five-row plan. every 1,000-game row plays the first 500 boards of `heldout-2p` in both seat orders (the boards of `831eea84`), so the rows pair game by game. canopy always sits in slot 0, so a record's score rate is canopy's share.

row	canopy plays	games	canopy's share	record	status
1	1600 sims vs our raw policy (`puffer-graph2/public@e3600`)	60 (cpu)	45.0 33.1–57.5	`00291a24`	stopped: blowout check only
1	the same, on the gpu build	1,000	—	`67e147c4` (to be written)	held
2	1600 sims vs our 2048-sim gumbel search on that network	1,000	33.0 30.2–36.0	`96d94fab`	done
3	its raw network (no search) vs our raw policy	1,000 (cpu)	30.5 27.7–33.4	`1c916131`	done
4	1600 sims vs F, the common reference	1,000	—	`9f0bafd7` (to be written)	held
5	1600 sims vs the deployed 1v1 bot (`BELIEF` on `puffer-graph2/full@e3350`, 16 worlds)	1,000	—	`aeaab0be` (to be written)	held

8.1 setup

8.1.1 the canopy bridge: fairness, cpu and gpu builds DONE

ran 2026-10-23 · **idea** marco

> **tl;dr.** released canopy plays in our own harness through a bridge to its rust search. the bridge shows canopy only what a fair player sees. the gpu build plays the same moves as the cpu build at 5.3 searched decisions a second against 0.2, which makes 1,000-game rows affordable.

question. can released canopy play in our harness fairly, under our rules, cheaply enough for 1,000-game rows?

setup. released canopy (`cullback/canopy` at `6185983`, nexus-v3 iteration 315) plays as a `CANOPY` agent: a bridge process to its rust search, built from v1's bridge. four patches sit under `third_party/canopy/` with a reproducible build. two are v1's: the per-move search budget and the bridge. two are ours: the bridge sends canopy only the count of the opponent's hidden dev cards (v1's sent the true cards), and tree reuse is exact. the rules are ours: 15 vp, discard above 9, no friendly robber, random dice. canopy trained with a friendly robber and balanced dice, so those inputs are new to it. it cannot express a one-card year of plenty (counted when offered). the gpu build is canopy's cuda backend on the same commit, patches and checkpoint.

result.

Table 33: checks on the canopy bridge. the cuda build was merged on its own as `b385ea9` (code ids unchanged).

check	result
hidden information	on 8,510 audited positions the bytes canopy received did not change when the opponent's hidden cards were redealt (in 2p the resource hands are public anyway). redealing victory point cards is audited too.
tree reuse	canopy re-roots only within its own turn and through forced moves; about half its decisions keep visits. less reuse than it gets natively, probably a small handicap.
smokes (4 games a row, plumbing only)	0 fallbacks, 0 moves our engine refused, 0 rule mismatches. canopy won 4 of 4 against our raw policy, twice.
cpu cost	5–7 cpu-seconds a decision at 1600 sims. the five-row cpu plan was about 14 h on 4 cores.
gpu vs cpu, same positions	on 300 positions at 1600 sims the gpu build picks the cpu build's move 300 of 300 times. root values differ by 2.5×10^{-4} on average, about 20 times less than changing the search seed. outputs do not depend on how many games are batched together.
gpu vs cpu, whole games	the same player statistically, not bit for bit: 29 of 40 smoke games identical, 2 winners changed. so gpu and cpu records key apart.
gpu throughput	4 bridges of 256 games each: about 5.3 searched decisions a second while sharing the gpu with the critic refit, against 0.2 for one cpu bridge.
gpu pinning	the cuda binary cannot be rebuilt byte for byte (a hash map in the kernel compiler), so the installed copy is the one the records name. it installs only as that pinned binary, checked by a behavioural fingerprint (64 fixed positions).

reading. the bridge is fair on hidden information and the gpu build is the same player as the cpu build. what still leans against canopy: our dice and robber rules, the one-card year of plenty, and half its native tree reuse.

reproduce

```
# cpu build: reproducible, gives the binary_sha256 pinned in
# third_party/canopy/canopy-bridge.manifest.json. SRC is a new directory; the checkpoint
# is model_iter_315.mpk of canopy's catan-nexus-v3 release
third_party/canopy/build_bridge.sh --backend cpu /tmp/canopy-src runs/_canopy model_iter_315.mpk
# cuda: not bit-reproducible, so keep the installed runs/_canopy-cuda
# (binary 1f7cc246...68802689); a rebuild is a new player for the record keys
```

8.2 results

8.2.1 canopy's raw network vs ours DONE

ran 2026-10-23 · **idea** marco · **config** `configs/experiments/canopy-bench-2p.yaml` · **records** `1c916131`

tl;dr. without search, canopy's network scores 30.5% against our raw policy over 1,000 games. our network is much stronger on its own, so canopy's case rests on its search.

question. network against network: how does canopy's policy without search do against our raw policy?

setup. row 3 of `canopy-bench-2p.yaml`, played as a standalone eval with no wandb mirror: canopy with `sims=0` (argmax over its legal moves) against `puffer-graph2/public@e3600` as a plain policy. 1,000 games, cpu build, 8 minutes.

result. canopy's share is **30.5** 27.7–33.4 (`1c916131`): 34% from the first seat, 27% from the second. 0 fallbacks; 15 one-card years of plenty it couldn't express.

reading. our raw policy is the stronger network by a wide margin. two things lean against canopy. its raw policy reads the opponent's unplayed dev cards as an expectation, where self-play encoded the exact hand (its search deals hands; its root input doesn't). and it never saw our dice or robber rules. its 1600-sim search is what the other rows test.

reproduce

```
OMP_NUM_THREADS=1 catanrl eval --suite heldout-2p --boards 500 --device cpu --threads 2 --num-envs 32 \
  "CANOPY:sims=0:procs=2" "POLICY:ckpt=puffer-graph2/public@e3600"
```

8.2.2 row 1 on cpu: canopy 1600 sims vs our raw policy, 60 games STOPPED

ran 2026-10-23 09:55–13:06 · **idea** marco · **config** `configs/experiments/canopy-bench-2p.yaml` · **wandb** canopy-bench-2p, eval only · **records** `00291a24`

> **tl;dr.** canopy with its released 1600-sim search scored 45.0% against our raw policy over 60 games. that only rules out a blowout either way. the 1,000-game gpu row (Section 8.2.4) replaces it.

question. does canopy's full search beat our raw policy?

setup. canopy at 1600 sims (`c_puct` 2.5, `c_visit` 50, `c_scale` 1, no dirichlet noise) against `puffer-graph2/public@e3600`, on the first 30 boards of `heldout-2p` in both seat orders, cpu build, 3 h 9 min. the cpu config sized its rows to canopy's cpu cost: 60 games for row 1, 40 for rows 2, 4 and 5, 1,000 for row 3. marco, rightly: "why would we play 60 games? how are we getting stat sig results with that?" 60 games is ±12.6 pp and 40 is ±15.5, so those rows could only detect a blowout. the gpu build (Section 8.1.1) makes 1,000-game rows affordable, and the cpu run stopped after this row.

result. canopy's share is **45.0** 33.1–57.5 (`00291a24`): 40% from the first seat, 50% from the second. 0 fallbacks, 0 refused moves.

reading. neither side crushes the other, and nothing finer can be read from 60 games. this is an underpowered row, kept only as a blowout check.

reproduce

```
# as launched: the cpu config plays row 1 first, then goes on to its 40-game rows
OMP_NUM_THREADS=1 nice -n 15 catanrl run configs/experiments/canopy-bench-2p.yaml
# row 1 alone
OMP_NUM_THREADS=1 catanrl eval --suite heldout-2p --boards 30 --device cpu --threads 2 --num-envs 32 \
  "CANOPY:sims=1600:c_puct=2.5:c_visit=50:c_scale=1:dirichlet_epsilon=0:procs=4" \
  "POLICY:ckpt=puffer-graph2/public@e3600"
```

8.2.3 row 2 on gpu: canopy 1600 sims vs our 2048-sim gumbel search DONE

ran 2026-10-23 13:41 – 2026-10-24 00:54 · **idea** marco · **config** `configs/experiments/canopy-search-2p-gpu.yaml` · **wandb** canopy-search-2p-gpu, eval only · **records** `96d94fab`

> **tl;dr.** canopy's full search against our full search, 1,000 games on the gpu build: canopy scores 33.0%, so our 2048-sim search wins two games in three. canopy is weakest in short games and even in the longest third.

question. search against search: canopy at 1600 sims against our raw public network inside the 2048-sim gumbel search of `831eea84` (Section 4).

setup. marco asked for this row soonest, so it runs on its own ahead of the other gpu rows. it has the same record id as row 2 of the full gpu config, which will find it and skip it. our side's spec resolves identically to `831eea84`'s. 1,000 games: ±3.1 pp, about 80% power for a 4–5 pp gap from 50%. the critic refit paused at e266 to free the gpu (Section 7).

result. canopy scores **33.0** 30.2–36.0 (`96d94fab`, n = 1,000; 36.8% from the first seat, 29.2% from the second). by game length it scores 18.0% in the shortest third (40–72 turns), 30.6% in the middle (72–85) and 50.3% in the longest (85–141). 0 fallbacks and 0 truncated games; once our engine refused a move canopy proposed (a reused tree's end turn) and a fresh search gave a legal one; 5 one-card years of plenty it couldn't express.

reading. our 2048-sim search beats canopy's 1600-sim search by more than it beats our own raw policy (62.4%, `831eea84`), and our raw network beats canopy's by a similar margin (69.5%, `1c916131`). so under our rules our bot is clearly stronger at both levels. canopy trained on colonist's 1v1 rules (friendly robber, balanced dice), which this harness doesn't play; losing the short games and holding even in long ones fits being worst early, where those rules matter most. that is a reading, not a test.

reproduce

```
CATANRL_CANOPY_DIR=$PWD/runs/_canopy-cuda OMP_NUM_THREADS=1 nice -n 5 \
  catanrl run configs/experiments/canopy-search-2p-gpu.yaml
```

8.2.4 rows 1, 5 and 4 on gpu, 1,000 games each PARKED

queued 2026-10-23 · **idea** marco · **config** `configs/experiments/canopy-bench-2p-gpu.yaml`

tl;dr. the rest of the benchmark at full size: canopy's search against our raw policy, the deployed 1v1 bot and F. about 19 h on a free gpu. held after row 2 (see status). each comparison can detect a gap of 2–5 pp.

question. is canopy's full search stronger than our raw policy and our deployed bot, and how does it fare against F next to our own bots?

setup. rows 1, 5 and 4 of Table 32 in that order, then row 2, which the config finds already played. 1,000 games each on `heldout-2p`'s first 500 boards, both seat orders. about 19 h on a free gpu, with the critic refit still paused: the two don't fit in 8 gb together. the config is on main since 2026-10-24. its records will be row 1 `67e147c4`, row 5 `aeaab0be`, row 4 `9f0bafd7`.

Table 34: the gap each comparison finds 80% of the time (two-sided 5%) at 1,000 games a row.

comparison	what it says	detectable gap
row 1 vs 50%	whether canopy's search beats our raw policy	≈ 4.4 pp
rows 1 vs 2, paired	what our search buys against canopy	≈ 5.2 pp
row 1 vs `1c916131`	what canopy's search buys over its raw network	≈ 5 pp
row 4 vs our policy's 95.6 vs F on the same boards	canopy against our policy, both measured on F	≈ 2.2 pp

status. held (2026-10-24). openhex's 1v1 queue plays friendly robber and balanced dice, canopy's training rules, so these rows may be more useful on those rules against the 1v1 bot retrained for them (Section 3). marco decides.

reproduce

```
# runtime.canopy_dir names runs/_canopy-cuda, so no CATANRL_CANOPY_DIR
OMP_NUM_THREADS=1 nice -n 5 catanrl run configs/experiments/canopy-bench-2p-gpu.yaml
```

9 monte carlo playouts instead of a tree

marco's idea: replace the search tree with plain playouts. for the policy's top k actions, play n full games from each with the raw policy on sampled worlds, and take the action with the best win rate. costed from our own numbers, it is far too slow as a player at k = 5, n = 1,000 (~150 gpu hours a 1,000-game row), but cheap enough as a reference that grades the search and the critic. nothing is queued.

9.1 monte carlo playouts over the top actions TODO

proposed 2026-10-23 · **idea** marco · **records** `831eea84`

tl;dr. costed, not run. as a player at k = 5, n = 1,000 it would cost ~150 gpu hours a 1,000-game row, against 3.9 h for mcts at 2048 sims, so not at that size. it is feasible as a reference q^π at a few hundred positions, or as a cheaper player with sequential halving or truncated playouts (both my suggestions, not queued).

idea. k ≈ 5 actions, n ≈ 1,000 playouts each, no critic at all. the policy plays fair (it sees only its own observation), so a playout in a world dealt from the tracker is a proper sample of the outcome. that makes this one step of policy iteration (tesauro's backgammon rollouts). the engine has the pieces: `cpp.sample_world` deals a world from a seat's tracker, and games copy.

cost.

Table 35: the cost of playouts as a player, from our own numbers.

quantity	value
decisions in a 2p game (`831eea84`)	~159, 81 of them the searching seat's
decisions left from a typical decision	~80
policy steps per decision at k = 5, n = 1,000	~400k
ppo rollout speed on the 2070 super	~42k decisions/s
time per decision / per game	~10 s / ~10 min
one 1,000-game harness row	~150 gpu h (mcts at 2048 sims: 3.9 h)
one decision on the bot vm	minutes

status. not viable as a player at that size. two cheaper uses:

1. **a reference.** q^π(s, a) for the top actions at a few hundred positions (500 positions ≈ 1.5 gpu hours), with common random numbers (the same worlds and dice for every action): ±1.6 pp per action at n = 1,000. it would grade the search's choices and the critic far better than one noisy 8192-sim search. (my suggestion, not queued)
2. **a cheaper player.** sequential halving over the actions, or truncated playouts (d moves, then the critic), at maybe 5–10× the cost of mcts at 2048. (my suggestion, not queued)

10 one model for any number of players and any map

marco's idea: train one network for any player count and any map, using masked numeric features and the graph network, which already handles any hex board. this section asks whether such a generalist can match the specialists. the answer so far: `ppo-mixed` comes within 2–3 points of each specialist against F and plays unseen blob maps nearly as well as BASE, but the specialists still win head to head, by about four points in 2p and three in 4p. training on blob maps too made it worse everywhere, so `ppo-mixed` is the bot on the island table (Section 13).

10.1 one network over 2–4 players

10.1.1 ppo-mixed: 2, 3 and 4 players on BASE and hexagon boards DONE

ran 2026-10-17 02:14–13:54 · **idea** marco · **config** `configs/experiments/ppo-mixed.yaml` · **wandb** ppo-mixed/main · **records** `b2690265`, `0f448f68`, `7c17d0ae`, `9b470a17`, `340ce1db`, `32aab9f3`

tl;dr. one network for 2, 3 and 4 players on two board types gets within 2–3 points of each specialist against F. it plays blob maps it never saw nearly as well as BASE (73.9 vs 77.0). head to head the specialists still win, by about four points in 2p and three in 4p.

question. can one network match the specialists at the same total number of decisions?

setup. `ppo-4p`'s recipe (Section 3) over six equally weighted scenarios: 2, 3 and 4 players on BASE and on `hexagon` boards (the BASE shape with shuffled numbers and generated harbors). the harness already does the masking. each row is padded to the env's maxima with a validity bit per slot, absent seats are all zero, the global block one-hots the number of seats, and the per-seat value head predicts every present seat. `env.scenarios` draws each game from a weighted list of (player count, rules, map generator). the budget is `ppo-4p`'s: 3800 epochs, 1B agent steps = 249M decisions, 11.5 h. blob-shaped boards of 15–30 tiles (`random-4p`) stay out of training because they would double the row (6316 floats vs 3202) and the memory; the run is tested on them zero-shot (`heldout-random-4p`). best checkpoint `e3550` by `select-mixed` (83.6). the run was launched without the wandb mirror and backfilled.

result.

Table 36: `ppo-mixed@e3550`. vs F rows are score shares (a win 1, a truncation 1/N). the 4p head to head is the corrected one: paired −2.7 pp −4.7–−0.8 per game for the generalist.

vs F	n	ppo-mixed	specialist	record
2p (`heldout-2p`)	5,000	92.3 91.6–93.0	95.2	`b2690265`
3p (the 3-player games of `heldout-mixed`; parity 33)	1,500	81.1 79.0–83.0	—	`9b470a17`
4p (`heldout-4p`)	4,000	77.0 75.7–78.3	78.6	`0f448f68`
blob maps, zero-shot (`heldout-random-4p`, 15–30 tiles)	4,000	73.9 72.5–75.3	—	`7c17d0ae`
the mixed suite as a whole (`heldout-mixed`)	4,500	79.8 78.6–80.9	—	`9b470a17`
head to head		**generalist**	**specialist**	
2p, vs `puffer-graph2`'s belief-averaged policy	5,000	45.7 44.4–47.1		`340ce1db`
4p, vs `ppo-4p` with two F, both slot orders (Section 10.1.2)	2 × 4,000	45.1	47.8	`32aab9f3`, `0903fae0`

reading. one network handles any seat count and board shape within 2–3 points of a specialist against F. the graph net transfers to board shapes it never trained on nearly for free (73.9 on blobs vs 77.0 on BASE). but every setting got a third of a specialist's decisions, and the specialists win the direct matches: by four points in 2p and three in 4p. so this is a budget question as much as a capacity one. the next step is to give the generalist more decisions per setting (Section 10.3.1).

reproduce

```
# training, then the six finals of eval.final
catanrl run configs/experiments/ppo-mixed.yaml
```

10.1.2 slot order decides 4p head-to-head tables DONE

ran 2026-10-18 16:00 · **config** `configs/experiments/slot-order.yaml` · **wandb** slot-order, eval only, ppo-4p/main (no local mirror), ppo-mixed/main · **records** `0903fae0`, `32aab9f3`, `b07e5bd9`

tl;dr. in a 4p table with two F, the slot right after an F loses about 12 points: two identical copies of `ppo-4p` split 40.45 / 52.75. so every 4p head to head has to run both slot orders. corrected that way, the 4p specialist beats `ppo-mixed` by 2.7 points, not 14.

question. the first 4p head to head had `ppo-mixed` at 39.3 against `ppo-4p`'s 53.0. how much of that gap is the seating?

setup. the suite seats `lineup[(seat + r) mod 4]`, so slot 0 always moves right after the F in slot 3. F (`enemies=next`) robs and blocks the seat after it. the diagnosis came from the tracker-modes table (Section 5). `slot-order` plays the swapped order [`ppo-4p`, `ppo-mixed`, F, F] on the same 1,000 boards × 4 rotations (4,000 games, one minute). averaging each agent over the two orders pairs them game by game.

result.

Table 37: 4p head-to-head shares with two F at the table, 4,000 games a record.

lineup (slots 0, 1; F in 2, 3)	slot 0	slot 1	record
`ppo-4p`, `ppo-4p` (identical copies)	40.45	52.75	`b07e5bd9`
`ppo-mixed`, `ppo-4p` (first reported)	39.3 37.8–40.9	53.0	`32aab9f3`
`ppo-4p`, `ppo-mixed` (swapped)	42.65	50.85	`0903fae0`
over both orders	`ppo-mixed` 45.1, `ppo-4p` 47.8: paired −2.7 pp −4.7–−0.8 per game		

reading. the 4p specialist does win, by three points rather than fourteen. every later 4p head to head runs both slot orders, or reports the two seats' sum, as the "two copies" row in Table 1 does.

reproduce

```
# the swapped order (0903fae0)
catanrl run configs/experiments/slot-order.yaml
# the identical-copies control (b07e5bd9, a ppo-4p final) and the first-reported order (32aab9f3, a ppo-mixed final)
catanrl eval "POLICY:ckpt=ppo-4p/main@e3650" "POLICY:ckpt=ppo-4p/main@e3650" F F --suite heldout-4p
catanrl eval "POLICY:ckpt=ppo-mixed/main@e3550" "POLICY:ckpt=ppo-4p/main@e3650" F F --suite heldout-4p
```

10.2 adding blob maps

10.2.1 ppo-mixed-blob: the generalist trained on blob maps too DONE

ran 2026-10-20 14:51 – 2026-10-21 15:02 · **idea** marco · **config** `configs/experiments/ppo-mixed-blob.yaml` · **wandb** ppo-mixed-blob/main · **records** `75cab037`, `799eac40`, `ac020b20`, `fb5a66f8`, `7c17d0ae`

tl;dr. adding blob maps as a seventh scenario made the generalist worse everywhere, the blobs included: 67.4 against `ppo-mixed`'s zero-shot 73.9. three things changed at once and none was isolated, so `ppo-mixed` stays the island bot.

question. zero-shot says the architecture copes with blob maps. does training on them beat `ppo-mixed`'s 73.9 on `heldout-random-4p`?

setup. `ppo-mixed` with `random-4p` as a seventh scenario of equal weight. same agent steps and optimizer steps: 512 games in flight over 7,600 epochs instead of 1,024 over 3,800, the same 32,768-row minibatch. the critic takes the lambda default (`ppo-mixed` trained with the older v-trace critic; Section 3). the blobs set the dims (30 tiles, 122 nodes, 151 edges, 611 actions, against 19 / 54 / 72 / 352), so every row is padded to them, and six of seven games are 19-tile boards run at blob size. memory is not the limit (5.5 GiB peak); compute is: 11.5k agent steps/s against `ppo-mixed`'s 24.7k, 24.2 h in all. best checkpoint `e7600`, the last, by `select-mixed` (78.8), still climbing at the end (76.8 at e7300).

result.

Table 38: `ppo-mixed-blob@e7600` against `ppo-mixed@e3550`, same suites and lineups.

vs F	n	ppo-mixed-blob	ppo-mixed	records (blob / mixed)
blob maps (`heldout-random-4p`)	4,000	67.4 65.9–68.8	73.9 72.5–75.3 (zero-shot)	`75cab037` / `7c17d0ae`
2p (`heldout-2p`)	5,000	90.1 89.3–90.9	92.3 91.6–93.0	`799eac40` / `b2690265`
4p (`heldout-4p`)	4,000	70.6 69.2–72.0	77.0 75.7–78.3	`ac020b20` / `0f448f68`
the mixed suite (`heldout-mixed`)	4,500	75.0 73.7–76.2	79.8 78.6–80.9	`fb5a66f8` / `9b470a17`

reading. a drop of 2–7 points in every setting, the added one included, is not the seventh scenario taking a seventh of the decisions instead of a sixth. something changed for all of them. three things did, none tested on its own: the lambda critic default, half as many games in flight per epoch, and every row padded to blob size. the deploy gate failed: for an island table `ppo-mixed` stays the stronger bot until one of those is isolated.

```
reproduce
catanrl run configs/experiments/ppo-mixed-blob.yaml
```

10.3 next

10.3.1 closing the gap to the specialists TODO

idea assistant

> **tl;dr.** give the generalist the specialists' per-setting budget, or fine-tune it per setting; if the gap closes, ship the generalist. separately, find which of `ppo-mixed-blob`'s three changes cost the 2–7 points. (my suggestion, not queued)

plan. every setting got a third of a specialist's decisions. either give the generalist the specialists' per-setting budget (3× the decisions, about 35 h) or fine-tune it per setting for a few hundred epochs. if the gap closes, the generalist is the model to ship. separately, rerun `ppo-mixed-blob` with one change at a time (the lambda critic, half the games in flight, padding to blob size) to find what cost the 2–7 points.

10.3.2 5–6 players on the extension map TODO

idea marco

> **tl;dr.** blocked on the engine: 5–6 player games need the special building phase. after that it is one more scenario in `ppo-mixed`.

status. not started. the engine lacks the special building phase. once it has one, 5–6 players are one more entry in `env.scenarios`.

11 trading

can a bot gain from domestic trades, and are they worth an action space? the engine has the rules, but the policy can't trade yet. a critic-only oracle finds few trades that help both sides: 2.5–6.2% of 4p positions at ≥ 2 pp, almost none in 2p. yet a harness that proposes every trade the critic calls good for both wins 45.5% of 4p games against a parity of 25. that is +20.5 pp paired, the largest effect in this log. the edge is in proposing, not accepting, it holds with the strongest 4p policy, and learned trading has not started.

11.1 is trading worth an action space?

11.1.1 trades in the action space and env TODO

idea marco

> **tl;dr.** the engine already plays domestic trades by catanatron's rules. the policy can't: the action space, the env and the row have no trades. this blocks a learned trader, not the harness below.

status. cppanatron implements domestic trades (offer, accept, reject, confirm, cancel) in parity with catanatron. what is missing is everything in front of it. the map-agnostic action space (`GraphActionSpace`) leaves trades out, the env never offers them, and the encoder has no trade prompt in the row. so "adding trading" means an action-

space extension plus the env, the mask and the trade prompts. the trade harness (Section 11.2.1) sidesteps all of it with one engine side channel (`VecEnv.set_action`).

11.1.2 trade oracle: which trades does the critic like for both sides? DONE

ran 2026-10-17 · **idea** marco

tl;dr. marco's first step: before any action-space work, ask the trained critic which trades help both sides. in 4p a trade worth ≥ 2 pp to both exists in 6.2% of positions; in 2p in 0.3%. worth an action space in 4p, not in 2p.

question. marco suspects that a policy trained with trades from scratch won't learn to trade well. so first: under the critics we already have, how often does a trade leave both parties better off, by how much, and what does it look like?

setup. `tools/trade_oracle.py`. 3,000 positions per network from its own self-play (temperature 1, about 3.4 positions per game, 884 games in 4p and 916 in 2p), taken after the roll. for each position, every 1:1, 2:1 and 1:2 trade between the mover and each opponent that both hands can pay (about 73 per 4p position), applied to a clone through the engine's own offer, accept and confirm actions. both seats' win probabilities are read from the mover's row: the critic trains on acting rows only, and a partner's own row is off-distribution. a seat's *gain* is the change in its win probability, in percentage points (pp). a trade is *mutual* at margin m if both parties gain at least m. the critic has a noise floor: its per-seat win probabilities don't conserve under a trade, with a median residual of 1.1 pp in 4p and 0.5 pp in 2p. so the 0.5 and 1 pp rows are within noise, and the 2 pp row is the one to read. there is no eval record and no wandb mirror; the numbers are in `report/data/trade-oracle-ppo-4p-main.json` and `report/data/trade-oracle-puffer-graph2-full.json`.

result.

Table 39: share of positions (%) with a mutual trade at each margin, 3,000 positions per critic.

critic	game, level	≥ 1 pp	≥ 2 pp	≥ 5 pp	residual (median)
`ppo-4p/main@best` (e3650)	4p, private	25.8 24.2–27.5	**6.2** 5.4–7.2	0.4	1.1 pp
`puffer-graph2/full@best` (e3350)	2p, full	4.3	0.3 0.1–0.5	—	0.5 pp

in 4p the best mutual trade gains the mover 1.2 pp and the partner 0.9 pp on average, and costs the two seats outside it 1.1 pp. 92% of mutual trades are 2:1 (the mover gives two); the most common is two wood for an ore. at the private level the mover's row sees the partner's hand only as a size, so the 2:1 skew is partly what the critic can see. the mover's selfish best trade gains it 7.8 pp and costs the partner 3.7 pp on average: trades are mostly zero-sum, as expected.

reading. 4p has a modest, real pool of 2:1 trades the critic likes for both sides, each paid for by the other two seats. two-player catan is nearly zero-sum for this critic, so a 2p trade action would almost never be used.

reproduce

```
# the defaults are the saved arguments; each writes report/data/trade-oracle-<run>.json
uv run python tools/trade_oracle.py --run ppo-4p/main          # 4p
uv run python tools/trade_oracle.py --run puffer-graph2/full   # 2p
```

11.1.3 the oracle on the calibrated lambda critic DONE

ran 2026-10-19 20:15

tl;dr. a rerun on the first critic whose four win probabilities sum to 1.00. the noise floor barely moves and the mutual pool shrinks to 2.5% of 4p positions at ≥ 2 pp. under this critic trades are nearly zero-sum between the parties, and the mutual gains sit at its noise floor.

question. does a calibrated critic give a cleaner oracle?

setup. the same tool on `aux-hidden-4p/plain@e600`, the first net trained with lambda critic targets (Section 3), whose per-seat win probabilities sum to the realized 1.00. 3,000 positions from 846 self-play games, 229k trades, 3 minutes. data in `report/data/trade-oracle-aux-hidden-4p-plain.json`; no wandb mirror.

result.

Table 40: the calibrated critic against the v-trace one: share of positions (%) with a mutual trade at each margin.

4p critic	residual median	residual p95	≥ 1 pp	≥ 2 pp	≥ 5 pp
v-trace (`ppo-4p/main@best`)	1.1 pp	4.8 pp	25.8 24.2–27.5	6.2 5.4–7.2	0.4
lambda (`aux-hidden-4p/plain@e600`)	1.0 pp	3.8 pp	22.7 21.2–24.3	**2.5** 2.0–3.2	0

the residual still reaches 2 pp in 90% of positions. the v-trace critic's sum sat at 0.93 and moved more under a hand change. under the lambda critic the best mutual trade gains the mover 0.8 pp and the partner 0.7 pp at the median. 94% of mutual trades are 2:1 (two sheep for a wheat, two wood for a wheat and two wood for a sheep lead the list), and the other two seats lose 1.1 pp on average. the mover's selfish best trade (+7.2 pp for it, −3.3 for the partner) is mutual in 0.1% of positions.

reading. the independence reference says what 2.5% means: if a trade's effect on the partner were unrelated to its effect on the mover, 77% of positions would show a mutual trade. so under this critic trades are nearly zero-sum between the two parties, and the mutual pool sits at the critic's own noise floor. a trade head would find a trade worth ≥ 2 pp to both in about one position in forty. a harness that proposes by this critic proposes gains the size of its noise, and resolving a one-point edge takes about 15,000 paired games. the harness below did far better than this predicts.

reproduce

```
uv run python tools/trade_oracle.py --run aux-hidden-4p/plain
```

11.2 a critic-driven trade harness

11.2.1 marco's trade harness: the pilot DONE

ran 2026-10-20 03:44–03:49 · **idea** marco · **config** `configs/experiments/trade-harness-pilot.yaml` · **wandb** trade-harness-pilot, eval only · **records** `0fc4352d`, `1d253765`, `c34e389c`, `09893ca0`

tl;dr. marco's idea: keep trades out of the action space and let the critic propose and answer them. in a 200-game pilot the proposing seat won 46.5% against three copies that only answer offers (parity 25), and 55.5% when it re-evaluates after every build. reading the true hands instead of sampled ones adds nothing.

question. marco's proposal after the oracle rerun: make the trade decision outside the action space, as a critic-driven harness that proposes the best mutual trades, with responders judging offers the same way. does a seat that does this win more?

setup. marco's rules: a calibrated critic, gains in each seat's own view, a pareto front for the proposer, positive expected value for the acceptor. the `TRADE` agent plays the policy everywhere. once per own turn, after the roll, it enumerates every 1:1, 2:1 and 1:2 bundle it can pay and reads the critic once for all of them (about 84 rows a read). it keeps the bundles whose gain is positive for it and for the partner, drops the pareto-dominated ones (those another bundle beats for both), and offers the rest in order of its own gain, up to three a turn. a responder accepts iff its mean gain over eight determinized worlds (hidden hands sampled consistent with what it has seen) is positive. the proposer confirms the acceptee best for itself.

trades go through one engine side channel (`VecEnv.set_action`); the row layout, the checkpoints and training are untouched. fairness is black-box: the choices are identical on any game with the same public state (`tests/test_trade_fairness.py`). native F never accepts an offer (its one-ply value sees no cards move on an accept, and a strict comparison keeps reject). so every table is one network against itself, `aux-hidden-4p/plain@e600` as policy and critic in every seat, with the trading seat the only asymmetry. four rows of 200 games (50 `heldout-4p` boards × 4 rotations), five minutes in all. parity is 25.

result.

Table 41: slot 0's share of wins against three responder-only copies on `heldout-4p`, 200 games a row.

proposer	share	offers / game	trades / game	record
marco's rule (0 pp margin)	**46.5** 39.7–53.4	14	12	`0fc4352d`
re-evaluates after every build	**55.5** 48.6–62.2	26	22	`1d253765`
1 pp margin, responders too	31.0 25.0–37.7	4	—	`c34e389c`
reads the true hands (unfair ceiling)	46.0 39.2–52.9	—	—	`09893ca0`

under marco's rule 85% of offers land, cancels are under 1%, and the three responders take 17–19. the bundles are the oracle's: mostly 1:1 and 2:1, with sheep for wood and ore for brick on top. so are the gains the critic claims for them (`tools/trade_pilot.py`): +1.35 pp for the mover per offer (median 1.2), +0.4 for the partner, +0.6 on an accepted read, −0.3 on a rejected one.

reading. sampling the hidden hands costs nothing: the true hands give the same share. the claimed gains sit at the critic's noise floor, yet a dozen small trades a game compound into twenty points of share, paid for by the seats outside each trade.

```
reproduce
catanrl run configs/experiments/trade-harness-pilot.yaml   # needs the aux-hidden-4p/plain run
# the bundles offered and the gains the critic claimed for them:
uv run python tools/trade_pilot.py --config configs/experiments/trade-harness-pilot.yaml --row 0
```

11.2.2 the harness at 4,000 games: m1 (0 pp margin) and m2 (1 pp margin) DONE

ran 2026-10-20 03:50–04:24 · **idea** marco · **config** `configs/experiments/trade-harness.yaml` · **wandb** trade-harness, eval only · **records** `ff0c2cb2`, `7ffdd60f`, `d7790a27`, `7c06e560`

tl;dr. at 4,000 games a row, marco's rule takes 45.5% against a parity of 25: +20.5 pp paired against the same table with nobody proposing ($p < 10^{-90}$). a 1 pp margin cuts that to +6.7 pp. the largest effect in this log.

question. does the pilot's edge hold at a sample size that resolves it, and does a margin above the critic's noise help?

setup. m1 is marco's rule; m2 puts the proposer and the responders at a 1 pp margin. each row has a control: the same games with a plain `POLICY` in slot 0, so nobody proposes and slots 1–3 are identical in both. 1,000 `heldout-4p` boards × 4 rotations = 4,000 games a row. the paired delta is slot 0's share against its control, with mcnemar's test on the games won only with trading or only without. same network and critic as the pilot; 34 minutes for the 16,000 games.

result.

Table 42: slot 0's share with trading against its no-proposer control, 4,000 games a row, `heldout-4p`. in brackets after the paired delta: games won only with trading / only without.

row	proposer	control	paired	offers, trades / game	records
m1, 0 pp	**45.5** 43.9–47.0	25.0 23.7–26.4	**+20.5 pp** (1,204 / 386), $p < 10^{-90}$	14.7, 12.4	`ff0c2cb2` / `7ffdd60f`
m2, 1 pp	31.7 30.3–33.2	25.0 23.7–26.4	+6.7 pp, $p < 10^{-14}$	3.8, 3.2	`d7790a27` / `7c06e560`

in m1, 84% of offers land and the responders take 18.0 / 18.3 / 18.3.

reading. under this critic the margin throws share away. the 0 pp rule trades four times as often as the 1 pp rule and wins three times as much over parity. so the critic's trade judgement carries signal well below its own per-read noise. the oracle was right about the per-trade gains and wrong about what a dozen of them a game add up to.

```
reproduce
catanrl run configs/experiments/trade-harness.yaml   # needs the aux-hidden-4p/plain run
```

11.2.3 m3: does accepting pay? DONE

ran 2026-10-20 05:31–06:38 · **config** `configs/experiments/trade-harness-m3.yaml` · **wandb** trade-harness-m3, eval only · **records** `fe7baed5`, `58f7c074`

tl;dr. no. at a table of three proposers, a seat that accepts the offers it likes does no better than one that rejects everything (12.7 against 13.2, $p = 0.54$). the proposer captures each trade's surplus: the edge is in proposing.

question. is the responder's side of a trade worth anything?

setup. three proposers at marco's rule. slot 0 is either the responder-only seat (accepts the offers its critic likes) or a plain `POLICY` that rejects every offer. 4,000 games a row, 8,000 in all, paired as in m1 (Section 11.2.2).

result.

Table 43: the acceptor against the rejecter, 4,000 games a row, `heldout-4p`.

slot 0, against three proposers	share	record
accepts the offers it likes	12.7 11.7–13.8	`fe7baed5`
rejects every offer	13.2 12.1–14.2	`58f7c074`

paired −0.45 pp: 371 games won only by the acceptor, 389 only by the rejecter, p = 0.54. the three proposers take 27–31.

reading. accepting buys nothing. the proposer picks the trades, so it captures each one's surplus, and the acceptor's own read of a gain is noise around zero. the passive seat is crushed whether it trades or not (13 against a parity of 25).

```
reproduce
catanrl run configs/experiments/trade-harness-m3.yaml   # needs the aux-hidden-4p/plain run
```

11.2.4 m4: the 4p specialist with trading DONE

ran 2026-10-20 14:35–14:51 · **config** `configs/experiments/trade-harness-m4.yaml` · **wandb** trade-harness-m4, eval only · **records** `c4900f87`, `7164d396`

tl;dr. the edge holds with the strongest 4p policy. `ppo-4p/main@best` in every seat, proposing by the calibrated critic, takes 41.2% (+16.2 pp paired), four points less than the weaker plain policy got in m1.

question. run because m1 was positive: does the trading edge survive a stronger player?

setup. m1's tables with `ppo-4p/main@best` (e3650) playing every seat. the calibrated critic of `aux-hidden-4p/plain@e600` judges the trades, because `ppo-4p`'s own v-trace critic does not sum to one. the control puts a never-trading `POLICY` in slot 0. 4,000 games a row.

result.

Table 44: `ppo-4p/main@best` with trading against three responder-only copies of itself, 4,000 games a row, `heldout-4p`.

slot 0	share	offers, trades / game	record
proposes (marco's rule)	**41.2** 39.7–42.7	13.8, 11.6	`c4900f87`
never trades (control)	25.0 23.7–26.4	—	`7164d396`

paired **+16.2 pp**: 1,080 games won only with trading against 433 only without, $p < 10^{-60}$.

reading. the edge holds, four points smaller than in m1. either a better player gets less from the same trades, or a critic from another network reads its positions slightly worse.

```
reproduce
catanrl run configs/experiments/trade-harness-m4.yaml   # needs the ppo-4p/main and aux-hidden-4p/plain runs
```

11.3 learned trading

11.3.1 a trade head trained from the ppo checkpoint TODO

idea marco

tl;dr. step (2) of marco's plan: a small trade head for the proposer and a yes/no for the responder, trained from the ppo checkpoint. not started; it needs trades in the action space first (Section 11.1.1).

plan. marco wants learned trading without an explosive action space. option one: a trade action plus 10 floats, one per resource for what the mover gives and what it gets, each rounded to an integer (11 extra outputs). option two: one output per integer amount (18 choices) per resource per side (181 extra outputs). a policy trained with trades from scratch probably won't learn to trade well. so the order is: first train a competent policy with the ppo recipe. then add option one for the proposer and a yes/no for the responder, trained from the ppo checkpoint with the trade actions masked in at a low entropy bonus. option one is the default.

open harness rows (not queued). the symmetric table, where every seat proposes; a number against F, which needs an F that answers offers; and whether a policy trained with trades on the table keeps the edge.

12 how markov a decision is

does the bot gain from remembering the game instead of seeing only the latest state? marco's scope runs from an lstm over the seat's own decisions (cheap) to a transformer over the whole history. the transformer costs orders of magnitude more compute, but in his view it is the only way to a "perfect" bot. it could learn, for example, that a seat which didn't play a knight while its best tile was blocked probably holds none. so far the lstm ties the markov graph net in 2p and beats it by 4.4 points head to head in 4p, and its memory is not reading the other hands. the transformer has not started.

12.1 an lstm over the seat's own decisions

12.1.1 the lstm policy, and the 2p test DONE

ran 2026-10-18 (built 2026-10-17) · **idea** marco · **config** `configs/experiments/arch-ablation.yaml` · **wandb** lstm, graph-s · **records** `1cd2de87`, `073da498`

> **tl;dr.** the cheap end of marco's scope: the graph net plus an lstm over the seat's own decisions. at 100M steps in 2p it ties the markov graph net (51.0 head to head, 90.3 against its 90.7 vs F): memory buys nothing measurable there.

question. does memory over the seat's own history help when the row already shows the latest state?

setup. preset `lstm`: the `graph-s` trunk, an lstm cell over its pooled embedding stepped at each of the seat's own decisions, and the memory fed into both heads; 0.86M parameters. the state persists across env steps (waiting rows keep it) and resets at game ends. ppo replays the recurrence over segments × horizon from the stored rollout states (cleanrl's ppo-lstm shape); a 5-epoch run at learning rate 0 reproduces the rollout's logits exactly (kl 0, clip fraction 0). the `POLICY` agent keeps one memory stream per seat, so records don't depend on `num_envs`. mcts, `BELIEF` and the server bridge refuse it, because a sampled world has no decision history. the test is the lstm cell of the formal arch ablation (Section 2): the same recipe as `graph-s`, 100M agent steps, one seed, `private` level, 5,000 `heldout-2p` games a row.

result. against F, the lstm takes 90.3 89.4–91.1 (`1cd2de87`) and `graph-s` 90.7 89.9–91.5. head to head, the lstm takes 51.0 49.7–52.4 (`073da498`). it trains at 14.8k steps/s against `graph-s`'s 17.6k (1.9 h); its select-2p best is e700 at 92.0. the full row is in the arch table (Section 2).

reading. at 100M steps in 2p, memory over the seat's own row history buys nothing measurable. the row already carries the played cards, the hand sizes and the robber history. separating the two would take the full 500M-step budget and more seeds.

reproduce

```
catanrl run configs/experiments/arch-ablation.yaml --only graph-s   # the head-to-head opponent
catanrl run configs/experiments/arch-ablation.yaml --only lstm
```

12.1.2 the lstm at 4p DONE

ran 2026-10-19 14:18–16:58 · **config** `configs/experiments/lstm-4p.yaml` · **wandb** lstm, markov control · **records** `9198cef1`, `5efc75ba`, `cf8368f4`, `d97f69b3`

> **tl;dr.** 4p hides more than 2p, so the same test at 4p. the lstm beats the same net without memory by 4.4 points head to head over both slot orders (8,000 games, one seed). against three F the two are inside the noise.

question. at 4p the row is missing a steal between two other seats, who played what when, and what a seat chose not to do. does memory pay there?

setup. the `lstm` preset on `ppo-4p`'s recipe at `aux-hidden-4p`'s budget: 750 epochs, 49.2M decisions, the lambda critic. the markov control is `aux-hidden-4p/plain@e600`, the `graph-s` cell with the same recipe, budget, seed and critic (Section 5). the head to head is one table with two F, played in both slot orders. `[A, B, F, F]` is not exchangeable: F robs the seat after it, so slot 0 loses about 10 points (Section 5). the paired reading averages the two orders. 4,000 games a row.

result.

Table 45: the lstm against the markov net at 4p, `heldout-4p`, shares in %. the slot after the F costs about 10 points (38.0 against 48.0 by slot); the lstm pays it in one order and the markov net in the other.

table	lstm	markov `graph-s`	record
three F (parity 25)	65.5 64.0–67.0	64.0 62.5–65.5	`9198cef1` / `d97f69b3`
lstm in slot 0, markov in slot 1, two F	39.7 38.1–41.2	45.3 43.8–46.9	`5efc75ba`
markov in slot 0, lstm in slot 1, two F	50.7 49.2–52.3	36.3 34.8–37.8	`cf8368f4`
both orders	**45.2**	40.8	paired **+4.4 pp** 2.4–6.3

the select curves sit within one half-width of each other at all fifteen points. the lstm's best is its last (e750 at 65.2), and it was still climbing (62.3, 61.8, 64.4, 65.2 from e600); the markov net's curve has been flat since e550. the lambda critic calibrates on the recurrent net as on the plain one: seat sum 1.59 at e21, 1.00 by e250, mean 1.000 over e300–750. cost: 22.3k steps/s, 27% slower than `graph-s`'s 30.5k; 2.53 h; peak 5.2 GiB.

reading. what bought nothing in 2p (51.0) buys four points at 4p against the same net without memory. the three-F row and the select curves don't separate them: the gain shows against the markov net, not against F. one seed. this takes the transformer off the shelf. the next question is what the memory carries (Section 12.1.3).

```
reproduce
catanrl run configs/experiments/aux-hidden-4p.yaml --only plain   # the markov control
PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True catanrl run configs/experiments/lstm-4p.yaml
```

12.1.3 does the lstm's memory read hands? DONE

ran 2026-10-19 17:00–19:41 · **idea** marco · **config** `configs/experiments/lstm-aux-4p.yaml` · **wandb** lstm + aux head, markov + aux head · **records** `722b7168`, `561c7ee2`, `f691b7fc`

tl;dr. marco's test: put the hidden-feature head on the lstm's memory and see whether it predicts the other hands better than the same head on the markov net. it doesn't (resources R^2 0.47–0.55 against 0.48–0.59). the memory's four points are not hand reading.

question. the lstm's expected edge at 4p is inferring the other hands from activity: what a seat built, what it chose not to build, who stole from whom. the row's tracker, a card counter, can't encode that. if the memory does it, a hand-regression head on the memory should beat the same head on the markov net.

setup. `lstm-4p` plus the hidden-feature head of Section 5 (coefficient 0.3, the `full` players block as the target) on the memory-injected embeddings. the reference is `aux-hidden-4p/aux`, the same head on the markov `graph-s` trunk at the same budget. R^2 per feature is over the last 50 epochs, from the logged `aux/mse_*` and `aux/var_*` ($R^2 = 1 - \text{mse} / \text{var}$, on wandb). nothing but the head reads the block, so the checkpoint plays as a fair `POLICY`. the recurrent aux path was built for this run (`tests/test_lstm_aux.py`, config schema 4). 2.58 h at 21.8k steps/s, peak 5.4 GiB.

result.

Table 46: what the hidden-feature head recovers about the other seats' hands, on the markov net (`aux-hidden-4p/aux`) and on the lstm's memory (`lstm-aux-4p`), last 50 epochs.

hidden feature (R^2)	markov head	memory head
resources, range	0.48–0.59	0.47–0.55
wood	0.55	0.47
wheat	0.59	0.55
ore	0.57	0.52
brick, sheep	0.48, 0.52	equal
actual vp	0.95	0.95
vp cards	0.65	0.63
knights	0.51	0.51
playable knight	0.42	0.42
road building	0.20	0.26
playable road building	0.19	0.24
head loss (lower is better)	0.017	0.020

the picture is the same at matched epochs 100, 300 and 500, so it is not a matter of training time. the policy takes 66.2 64.7–67.6 against three F (`722b7168`; `lstm-4p` took 65.5), select best e750 at 68.4. head to head against `lstm-4p/main@e750`, with two F and both slot orders (`561c7ee2`, `f691b7fc`, 8,000 games): 43.8 against 42.3, paired +1.5 pp −0.4–3.5.

reading. the memory adds no hand information beyond the tracker row: whatever the recurrent state carries, the head can't extract more of the other seats' cards from it. the aux-shaped memory doesn't move the policy either, just as the head didn't move the markov policy. one coefficient was tried (0.3, set on the 2p graph net, not re-measured for the lstm). what the memory does carry is open: the order of events, or who did what to whom.

reproduce

```
catanrl run configs/experiments/aux-hidden-4p.yaml --only aux   # the markov head's R² reference
# after lstm-4p, whose checkpoint the finals play against:
PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True catanrl run configs/experiments/lstm-aux-4p.yaml
```

12.2 the whole history

12.2.1 a transformer over the whole history TODO

idea marco

tl;dr. the expensive end of marco's scope. it was parked after the 2p tie; the 4p lstm's +4.4 pp takes it off the shelf. not started.

status. a sequential model over the entire game history costs orders of magnitude more compute than the markov net. the plan was to build it only if the lstm beat the markov net, and at 4p it did. the question to answer first is what the memory carries, since it isn't hands: the candidates are the order of events and who did what to whom.

13 deployment

this section records which bots play people, on catan.marcoya.ng and on openhex, and why each was picked. the 1v1 bot is `BELIEF` on `puffer-graph2/full@e3350` with 16 worlds; the fair `public` network (`puffer-graph2/public@e3600`) plays as well with one forward pass, no sampled worlds and no tracker, so it is the bot to serve raw (Section 3). a third table seats the generalist `ppo-mixed` on random islands. search reuse is available to the deployed bots but neutral in strength.

Table 47: what is deployed, as `deploy/house.yaml` and `deploy/openhex.yaml` list it in this checkout. `@best` is e3350 for `puffer-graph2/full`, e3650 for `ppo-4p/main` and e3550 for `ppo-mixed/main`.

where	table or bot	agent spec
catan.marcoya.ng	"1v1 vs puffer-graph2"	`BELIEF:ckpt=puffer-graph2/full@best:worlds=16`
catan.marcoya.ng	"4p vs ppo-4p"	`POLICY:ckpt=ppo-4p/main@best`
catan.marcoya.ng	"4p island vs ppo-mixed" (15–30 tile islands)	`POLICY:ckpt=ppo-mixed/main@best`
openhex	1v1	`BELIEF:ckpt=puffer-graph2/full@best:worlds=16`
openhex	4p	`POLICY:ckpt=ppo-4p/main@best`
openhex	1v1, search	`MCTS:ckpt=puffer-graph2/full@best:sims=1024:dets=1`

13.1 tables on catan.marcoya.ng

13.1.1 the 4p island table DONE

ran 2026-10-22 18:59 utc (live) · **idea** marco · **config** `deploy/house.yaml` · **wandb** ppo-mixed, ppo-mixed-blob · **records** `7c17d0ae`, `75cab037`

tl;dr. catan.marcoya.ng has a third table, "4p island vs ppo-mixed": the generalist `ppo-mixed@best` on a fresh 15–30 tile island in every room. it beat the blob-trained candidate on random islands, 73.9 against 67.4 versus F. marco played two recorded games that evening: both verified, no fallbacks, and the bots won both.

question. which network should play a table of random island maps?

setup. two candidates on `heldout-random-4p` (blob-shaped boards of 15–30 tiles) against three F. `ppo-mixed` never trained on islands; `ppo-mixed-blob` did, and its deploy gate failed (Section 10). the island table runs next to the two existing tables (Table 47). before going live, a preflight and a loopback smoke ran on the bot vm's own stack: two island games, all verified, 11 ms a decision.

result.

Table 48: the two island candidates on `heldout-random-4p`, 4,000 games each, parity 25.

candidate	vs three F (share)	record
`ppo-mixed/main@e3550`, zero-shot on islands	**73.9** 72.5–75.3	`7c17d0ae`
`ppo-mixed-blob/main@e7600`, trained on islands	67.4 65.9–68.8	`75cab037`

marco's two recorded games on the live table were both verified, with no fallbacks; the bots won both.

reading. `ppo-mixed` is the stronger island bot until one of the blob run's three changes is isolated (Section 10).

```
reproduce
catanrl eval "POLICY:ckpt=ppo-mixed/main@e3550" F F F --suite heldout-random-4p
catanrl eval "POLICY:ckpt=ppo-mixed-blob/main@e7600" F F F --suite heldout-random-4p
catanrl house deploy/house.yaml   # on the bot vm: keeps the three tables open
```

13.2 search in the deployed bots

13.2.1 search reuse on the server bridge and the openhex seat DONE

ran 2026-10-20 · **idea** marco

> **tl;dr.** search reuse is merged into the server bridge: a deployed seat's search can survive its own moves. it is opt-in, and neither deploy file turns it on. the options are neutral in strength (Section 4.3.1), so the deployed bots can take them or leave them.

question. the deployment half of marco's ask for tree reuse and leaf deduplication: can a deployed seat keep its search between its own moves?

setup. the server bridge keeps an `MCTS` seat's search between its decisions when the spec sets `reuse=true`. one fix came with the openhex seat's wiring: its robber move after its own discards re-rooted a tree dealt before the discards were known (three or more players, normal robber). it now searches fresh there.

result. merged. reuse is opt-in per spec, and neither `deploy/house.yaml` nor `deploy/openhex.yaml` sets it. on the cpu the transposition table adds 0.2–0.5 s to a fresh 1024-sim search.

reading. the deployed bots can turn reuse on at no measurable cost in strength, but at 1024 sims it buys none either (Section 4.3.1).

```
reproduce
# no eval of its own: the strength rows are in the search-options block. to serve reuse, append
# :reuse=true to an MCTS spec in deploy/openhex.yaml, e.g.
#   MCTS:ckpt=puffer-graph2/full@best:sims=1024:dets=1:reuse=true
catanrl openhex deploy/openhex.yaml
```

14 llm env and eval

can catan serve as an env and an eval for language models? marco wants frontier models evaluated as players, and eventually a small model trained with rl on the game. the eval half is built and tested with a free stub model, but no real model has played: it waits on an api key and a budget. rl is not possible on this box's 8 GB gpu.

14.1 frontier models as players

14.1.1 llm players in the harness TODO

built 2026-10-17 · **idea** marco · **config** `configs/experiments/llm-eval.yaml`

> **tl;dr.** an `LLM` agent plays through the catanatron harness with catanllm's five prompting strategies, on public information only. a 16-cell sweep is ready. a 50-game cell costs about $16–26 as a lower bound; it needs an openrouter key and a budget.

setup. `catanllm` (the cs 159 project) already benchmarks llm players through openrouter against catanatron bots with a few prompting strategies. v2 ports it as an agent spec, `LLM:model=<openrouter id>:strategy=plan|basic|strategy|search|rules:thinking=true`, that plays through the catanatron harness next to `POLICY` and F. the prompt holds public information only: opponents appear as hand sizes, dev-card counts, played cards and visible points (the original renderer leaked every seat's dev cards; a test now pins that). a stub model plays the same games for

free, so the whole path is tested without a key. real-model records are marked non-replayable, so a report refresh can't quietly spend money. v2's game server (protocol 5: committed seeds, signed moves, verifiable transcripts) can also seat an llm client against our bots fairly.

plan. `llm-eval.yaml` is a 16-cell sweep: four models × plan or basic prompting × against F or against `puffer-graph2/full@best`'s policy, on the first 20 `heldout-2p` boards from both seats. the model ids are placeholders. `catanrl llm-cost` prices it from stub games: a 2p game to 15 points prompts the model about 20 times at 2–8k tokens a prompt, so a 50-game cell costs about $16–26 at gpt-5 or sonnet-4.5 prices, as a lower bound. a model that plays well lasts longer and is prompted 70–90 times a game, so budget 3–4× that. no real-model run yet, so no records and no wandb run.

reproduce

```
catanrl llm-cost --games 5 --strategies plan,basic --price gpt-5=1.25,10   # the budget, free
catanrl run configs/experiments/llm-eval.yaml --smoke                       # every cell with stub models, free
OPENROUTER_API_KEY=... catanrl run configs/experiments/llm-eval.yaml        # every cell, real calls cost money
```

14.2 catan as an rl env for language models

14.2.1 rl a small language model on catan TODO

idea marco

> **tl;dr.** the other half of marco's idea: train a small language model with rl in this env (for example as a harbor task). deferred: it doesn't fit on this box.

status. deferred. rl on a language model needs more than the 8 GB gpu this repo runs on; the eval above is the cheap half.

14.2.2 an llm env for building catan bots TODO

idea marco

> **tl;dr.** turn "make a strong catan bot" itself into an llm rl env and eval. marco expects the next frontier models could get very good at building better catan bots.

status. an idea; nothing is built.